Final_project

March 11, 2026

1 Final Project: "Machine Learning and Analytics: Surviving the Berry Island"

Student: s-001073 (Abhishek Halivan) https://github.com/Abhishek-Halivan/Final_Project.git

2 Final Project Problem statement

You find yourself on an island that you have never seen before. You cannot remember how you got here, but your rations will only last until Wednesday, September 10th. Luckily, the island seems to be of fertile soil, and full of mysterious berry bushes.

1. You carefully collect your notes on the local berries in a file called s-001073_clustering.json. You proceed to try to make sense of the diversity of different berries growing here by grouping them into clusters.
    1. Use the clustering algorithms we covered in the course to cluster the data that is presented into groups.
    2. What clustering do you obtain when using different features or different numbers of clusters?
    3. Can you obtain clearer clusters if you incorporate derived variables, such as the growth rate (height/age)?
2. Through some painful trial and error, you have discovered that some of the berries are edible and others are not. You can tell them apart by their leaf shape, petal, and berry colors. You made notes of these features in s-001073_edible.json.
    1. Can you identify the rules by which you can determine whether a plant is edible?
    2. You located some notes by a botanical expedition that was here before you. They identified five different species of berry, which you duly enter into your fact sheet. Can you distinguish the species given your information? This is recorded in s-001073_species.json.
    3. Unfortunately, the bushes rarely have more than one of leaves, petals, and berries. Can you determine their species, or at least whether they are edible or not, using only one of these characteristics (plus whatever else you know about the plant)? Compare the effectiveness of only looking at "edibleness" as the target variable compared to deriving a species first and then using your knowledge of which species are edible. The corresponding subsets of your notes have been recorded in s-001073_berry.json, s-001073_leaf.json, and s-001073_petal.json.
3. Given that berries are your main food source, it would be beneficial to predict how large a plant's average berry will be given what you know about the plant.
    1. Considering everything you know about the plant apart from the size of its berries, predict the berry size using different methods (use s-001073_complete.json).

2. Does the prediction improve if you only perform it for edible berries (since, in all honesty, who cares about the size of inedible berries)?
3. Derive an explicit expression expressing the expected berry size in terms of the remaining features. This way, you can also use your predictor out in the wilderness, where no computer is available.
4. How well can you predict the size of the berry using only a subset of the total features? For instance, the quality of the soil can be very difficult to assess for non-experts, so it would be great to predict the size of the berry without it.

Instructions

For each of the exercises, provide:

1. a clearly written answer in English (or German) that answers the question;
2. code that you have executed in the process what you have been doing;
3. plots and/or other statistics that support your arguments; Where appropriate, you are encouraged to make connections between your findings and the theory as presented in the lectures.

Please submit your solutions as a Jupyter notebook or pdf file. If you choose to use a pdf file, you may either upload your code separately in one or more Python script files or include the code in your PDF (preferably in a dedicated font/writing environment for clarity).

Hints You can do the exercises in any order (Remember: You are not actually stuck on an island of berries).

3 Final Project Solution - Berry Island Survival Guide

So here I am, stuck on some island I've never seen before. No idea how I got here, but my rations run out on September 10th. The good news? There are tons of berry bushes everywhere.

Plan: 1. **Clustering** - Group the berry plants to understand what kinds of plants are out there 2. **Classification** - Figure out which berries are safe to eat and identify species 3. **Regression** - Predict how big the berries will be so I can prioritize the best food sources

[78]:
```
# Import everything we need
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
import os
import warnings
warnings.filterwarnings('ignore')

from sklearn.preprocessing import StandardScaler, LabelEncoder,␣
↪PolynomialFeatures
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import classification_report, confusion_matrix
```

```
from sklearn.metrics import accuracy_score, mean_squared_error,␣
↪mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split, cross_val_score,␣
↪cross_val_predict, LeaveOneOut
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LinearRegression
from sklearn.decomposition import PCA

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

print("All libraries loaded successfully!")
```

```
All libraries loaded successfully!
```

[79]:
```
# Load all the datasets
data_dir = os.path.join('..', 'dataset')

df_clustering = pd.DataFrame(json.load(open(os.path.join(data_dir,␣
↪'s-001073_clustering.json'))))
df_edible = pd.DataFrame(json.load(open(os.path.join(data_dir, 's-001073_edible.
↪json'))))
df_species = pd.DataFrame(json.load(open(os.path.join(data_dir,␣
↪'s-001073_species.json'))))
df_complete = pd.DataFrame(json.load(open(os.path.join(data_dir,␣
↪'s-001073_complete.json'))))
df_berry = pd.DataFrame(json.load(open(os.path.join(data_dir, 's-001073_berry.
↪json'))))
df_leaf = pd.DataFrame(json.load(open(os.path.join(data_dir, 's-001073_leaf.
↪json'))))
df_petal = pd.DataFrame(json.load(open(os.path.join(data_dir, 's-001073_petal.
↪json'))))

# Quick overview
datasets = {
    'clustering': df_clustering, 'edible': df_edible, 'species': df_species,
    'complete': df_complete, 'berry': df_berry, 'leaf': df_leaf, 'petal':␣
↪df_petal
}

for name, df in datasets.items():
    print(f"\n{'='*50}")
    print(f"Dataset: {name} | Shape: {df.shape}")
    print(f"Columns: {list(df.columns)}")
    print(df.head(3))
```

```
    print()
```

```
==================================================
Dataset: clustering | Shape: (500, 6)
Columns: ['age', 'leaf_size', 'berry_size', 'petal_size', 'soil_quality',
'height']
        age  leaf_size  berry_size  petal_size  soil_quality      height
0  2.146854  47.780380    9.079037    3.184050      5.224060   96.043558
1  4.265461  36.554632    9.533910    3.888529      5.202135  143.901331
2  0.216002  44.545243    0.000000    3.403425      4.792088   10.323533


==================================================
Dataset: edible | Shape: (500, 4)
Columns: ['leaf_shape', 'petal_colour', 'berry_colour', 'edible']
   leaf_shape petal_colour berry_colour  edible
0       ovate        white        green   False
1  lanceolate        white         grey   False
2       ovate         pink         none   False


==================================================
Dataset: species | Shape: (500, 4)
Columns: ['leaf_shape', 'petal_colour', 'berry_colour', 'species']
   leaf_shape petal_colour berry_colour              species
0       ovate        white        green       berrius codiae
1  lanceolate        white         grey      berrius elephas
2       ovate         pink         none  berrius delphinorum


==================================================
Dataset: complete | Shape: (500, 11)
Columns: ['age', 'leaf_size', 'berry_colour', 'berry_size', 'leaf_shape',
'petal_colour', 'petal_size', 'height', 'edible', 'soil_quality', 'species']
        age  leaf_size berry_colour  berry_size  leaf_shape petal_colour  \
0  2.146854  47.780380        green    9.079037       ovate        white
1  4.265461  36.554632         grey    9.533910  lanceolate        white
2  0.216002  44.545243         none    0.000000       ovate         pink

   petal_size      height  edible  soil_quality              species
0    3.184050   96.043558   False      5.224060       berrius codiae
1    3.888529  143.901331   False      5.202135      berrius elephas
2    3.403425   10.323533   False      4.792088  berrius delphinorum


==================================================
Dataset: berry | Shape: (500, 7)
```

```
Columns: ['age', 'height', 'soil_quality', 'berry_colour', 'berry_size',
'edible', 'species']
        age      height  soil_quality berry_colour  berry_size  edible  \
0  2.146854   96.043558      5.224060        green    9.079037   False
1  4.265461  143.901331      5.202135         grey    9.533910   False
2  0.216002   10.323533      4.792088         none    0.000000   False

               species
0       berrius codiae
1      berrius elephas
2  berrius delphinorum


==================================================
Dataset: leaf | Shape: (500, 7)
Columns: ['age', 'height', 'soil_quality', 'leaf_shape', 'leaf_size', 'edible',
'species']
        age      height  soil_quality  leaf_shape  leaf_size  edible  \
0  2.146854   96.043558      5.224060       ovate  47.780380   False
1  4.265461  143.901331      5.202135  lanceolate  36.554632   False
2  0.216002   10.323533      4.792088       ovate  44.545243   False

               species
0       berrius codiae
1      berrius elephas
2  berrius delphinorum


==================================================
Dataset: petal | Shape: (500, 7)
Columns: ['age', 'height', 'soil_quality', 'petal_colour', 'petal_size',
'edible', 'species']
        age      height  soil_quality petal_colour  petal_size  edible  \
0  2.146854   96.043558      5.224060        white    3.184050   False
1  4.265461  143.901331      5.202135        white    3.888529   False
2  0.216002   10.323533      4.792088         pink    3.403425   False

               species
0       berrius codiae
1      berrius elephas
2  berrius delphinorum
```

---

4 Part 1: Clustering the Berry Plants

4.1 Task 1.1 - Clustering the Complete Dataset

I started by trying to group the 500 berry plants using two different clustering approaches: K-Means (distance-based) and Gaussian Mixture Model (EM algorithm). I used all the numeric features from the clustering dataset (age, height, soil_quality, leaf_size, petal_size, berry_size) and standardized them first so no feature dominates just because of its scale. I tried K-Means with k=3 (since there are 5 species, I thought maybe 3-5 clusters would make sense) and GMM with 3 components using the EM algorithm.

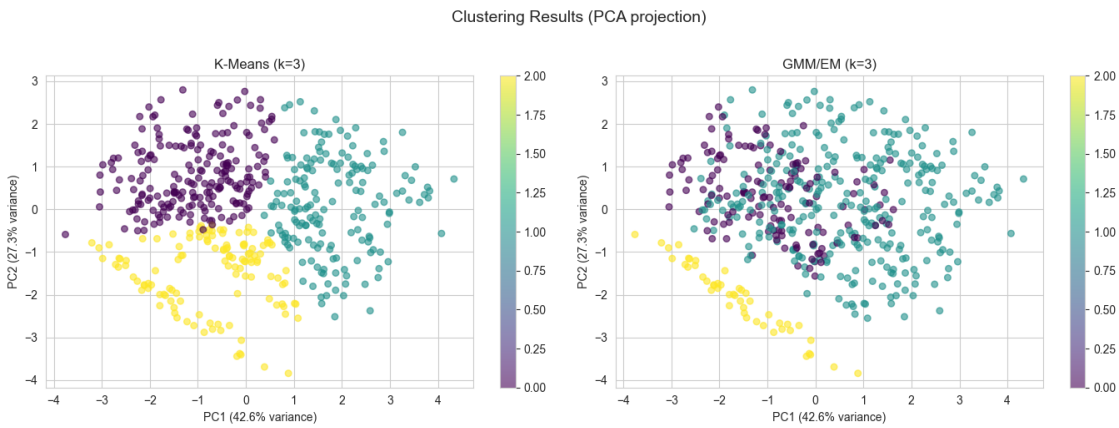
[80]:
```
# Task 1.1: Clustering with K-Means (distance-based) and GMM (EM algorithm)

# Step 1: Standardize the features (super important for clustering!)
scaler = StandardScaler()
X_cluster = df_clustering.copy()
X_scaled = scaler.fit_transform(X_cluster)

print("Original data stats:")
print(df_clustering.describe().round(2))
print("\nAfter scaling, all features have mean~0 and std~1")
print(f"Scaled data shape: {X_scaled.shape}")

# K-Means (distance-based clustering) with k=3 as a starting point
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Gaussian Mixture Model (EM algorithm) with 3 components
gmm = GaussianMixture(n_components=3, random_state=42, n_init=5)
gmm.fit(X_scaled)
gmm_labels = gmm.predict(X_scaled)
gmm_probs = gmm.predict_proba(X_scaled)

# Evaluate: K-Means uses inertia (within-cluster sum of squares)
print(f"\nK-Means Evaluation:")
print(f"  Inertia (SSE): {kmeans.inertia_:.2f}  (lower = tighter clusters)")

# GMM gives us BIC and AIC for model selection
print(f"\nGMM/EM Evaluation:")
print(f"  BIC: {gmm.bic(X_scaled):.2f}  (lower = better model)")
print(f"  AIC: {gmm.aic(X_scaled):.2f}")
print(f"  Log-likelihood: {gmm.score(X_scaled):.4f}")

# Show the soft assignments (probabilities) for first 5 points
print(f"\nGMM soft cluster assignments (first 5 points):")
print(f"  Each row shows the probability of belonging to each cluster:")
for i in range(5):
    probs = ', '.join([f'{p:.3f}' for p in gmm_probs[i]])
```

```
    print(f"  Point {i}: [{probs}] -> assigned to cluster {gmm_labels[i]}")
```

```
Original data stats:
          age  leaf_size  berry_size  petal_size  soil_quality  height
count  500.00     500.00      500.00      500.00        500.00  500.00
mean     4.71      32.31        6.10        2.60          4.53  170.55
std      2.81       9.93        2.69        0.86          0.85  137.58
min      0.01      11.58        0.00        0.61          3.00    0.29
25%      2.27      25.24        4.84        1.96          3.84   60.75
50%      4.59      31.62        6.51        2.63          4.58  132.59
75%      7.08      38.59        7.94        3.21          5.27  248.00
max      9.94      59.78       12.49        4.59          5.99  670.11

After scaling, all features have mean~0 and std~1
Scaled data shape: (500, 6)

K-Means Evaluation:
  Inertia (SSE): 1728.51  (lower = tighter clusters)

GMM/EM Evaluation:
  BIC: 5803.06  (lower = better model)
  AIC: 5453.24
  Log-likelihood: -5.2872

GMM soft cluster assignments (first 5 points):
  Each row shows the probability of belonging to each cluster:
  Point 0: [0.000, 1.000, 0.000] -> assigned to cluster 1
  Point 1: [0.000, 1.000, 0.000] -> assigned to cluster 1
  Point 2: [0.000, 0.000, 1.000] -> assigned to cluster 2
  Point 3: [0.000, 0.000, 1.000] -> assigned to cluster 2
  Point 4: [0.000, 1.000, 0.000] -> assigned to cluster 1
```

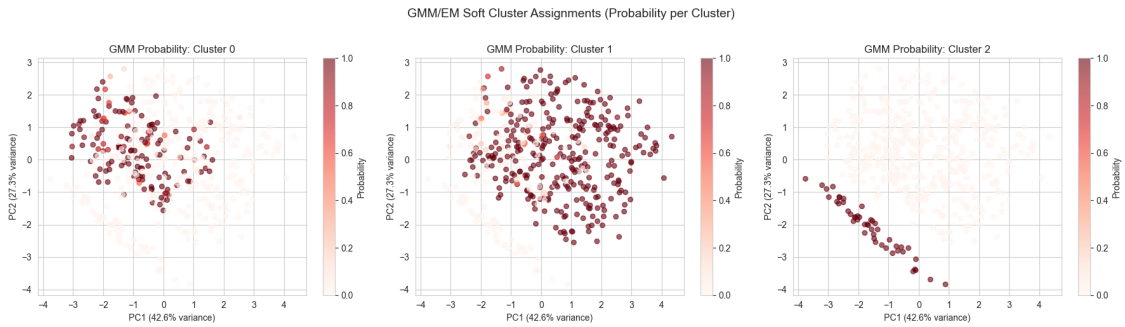
[81]:
```
# Visualize the clustering results using PCA to reduce to 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, labels, title in zip(axes,
    [kmeans_labels, gmm_labels],
    ['K-Means (k=3)', 'GMM/EM (k=3)']):
    scatter = ax.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis',␣
 ↪alpha=0.6, s=30)
    ax.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
    ax.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
    ax.set_title(title)
    plt.colorbar(scatter, ax=ax)
```

```
plt.suptitle('Clustering Results (PCA projection)', fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig('plot_1_1_clustering_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_1_1_clustering_comparison.png")
```




```
Saved: plot_1_1_clustering_comparison.png
```

[82]:
```
# Visualize GMM soft assignments - the key advantage of EM over K-Means
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for j in range(3):
    scatter = axes[j].scatter(X_pca[:, 0], X_pca[:, 1], c=gmm_probs[:, j],
                              cmap='Reds', alpha=0.6, s=30, vmin=0, vmax=1)
    axes[j].set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
    axes[j].set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
    axes[j].set_title(f'GMM Probability: Cluster {j}')
    plt.colorbar(scatter, ax=axes[j], label='Probability')

plt.suptitle('GMM/EM Soft Cluster Assignments (Probability per Cluster)',␣
 ↪fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig('plot_1_1_gmm_probabilities.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_1_1_gmm_probabilities.png")
print("\nUnlike K-Means which gives hard assignments, GMM/EM shows how␣
 ↪'uncertain' each point is.")
print("Points near cluster boundaries have split probabilities - very useful␣
 ↪information!")
```




```
Saved: plot_1_1_gmm_probabilities.png

Unlike K-Means which gives hard assignments, GMM/EM shows how 'uncertain' each
point is.
Points near cluster boundaries have split probabilities - very useful
information!
```

Results: - K-Means (k=3): Inertia (SSE) 1729 — measures how tight the clusters are - GMM/EM (k=3): BIC 5803, AIC 5453

My thoughts: The clusters don't seem super distinct — looking at the PCA visualizations, the groups overlap quite a lot. K-Means gives hard assignments (each point belongs to exactly one cluster), while GMM gives soft probabilistic assignments. The BIC and AIC from GMM will be really useful in Task 1.2 to figure out the best number of clusters.

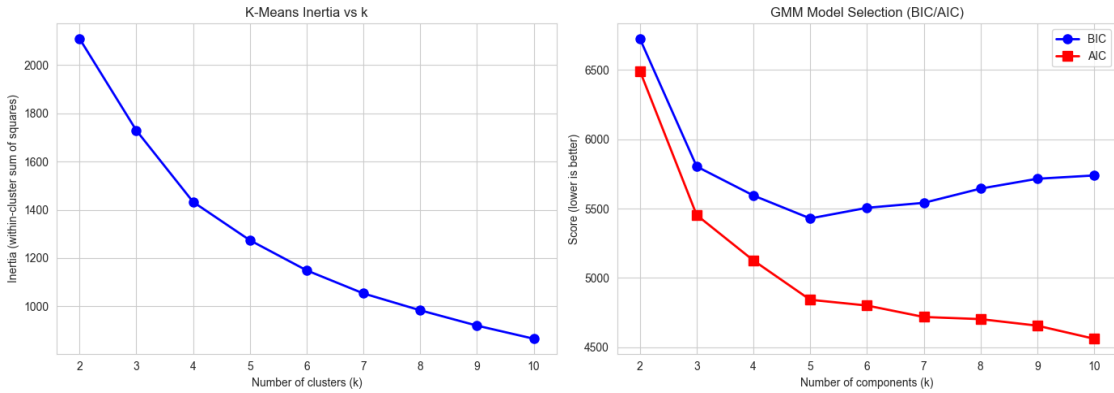
But the cool thing about GMM is that it gives us *soft* cluster assignments — instead of just saying "this point belongs to cluster 2," it says "this point has 75% probability of being in cluster 2 and 25% in cluster 1." That extra information could be useful when a plant doesn't clearly belong to one group. The BIC and AIC scores from GMM help us decide the optimal number of components (I'll explore this more in Task 1.2).

4.2 Task 1.2 - Different Features and Number of Clusters

Next I explored whether using different subsets of features or different numbers of clusters would give better results. I used both K-Means (distance-based) and GMM (EM algorithm) to compare the two approaches.

[83]:
```
# Task 1.2: K-Means inertia and GMM BIC/AIC for optimal k

k_range = range(2, 11)
inertias = []
bics = []
aics = []

for k in k_range:
    # K-Means
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
```

```
    km.fit(X_scaled)
    inertias.append(km.inertia_)

    # GMM (EM)
    gm = GaussianMixture(n_components=k, random_state=42, n_init=5)
    gm.fit(X_scaled)
    bics.append(gm.bic(X_scaled))
    aics.append(gm.aic(X_scaled))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Inertia plot (K-Means) - lower = tighter clusters
axes[0].plot(k_range, inertias, 'bo-', linewidth=2, markersize=8)
axes[0].set_xlabel('Number of clusters (k)')
axes[0].set_ylabel('Inertia (within-cluster sum of squares)')
axes[0].set_title('K-Means Inertia vs k')
axes[0].set_xticks(list(k_range))

# BIC/AIC for GMM - lower = better model
axes[1].plot(k_range, bics, 'bo-', linewidth=2, markersize=8, label='BIC')
axes[1].plot(k_range, aics, 'rs-', linewidth=2, markersize=8, label='AIC')
axes[1].set_xlabel('Number of components (k)')
axes[1].set_ylabel('Score (lower is better)')
axes[1].set_title('GMM Model Selection (BIC/AIC)')
axes[1].set_xticks(list(k_range))
axes[1].legend()

plt.tight_layout()
plt.savefig('plot_1_2_inertia_bic.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_1_2_inertia_bic.png")

best_k_km = list(k_range)[np.argmin(inertias)]
best_k_gmm = list(k_range)[np.argmin(bics)]
print(f"\nK-Means: Inertia keeps decreasing (from {inertias[0]:.0f} at k=2 to␣
↪{inertias[-1]:.0f} at k=10)")
print(f"  - more clusters always gives lower inertia, but too many clusters␣
↪overfits")
print(f"GMM: Lowest BIC at k={best_k_gmm} - BIC balances model fit with␣
↪complexity")
```

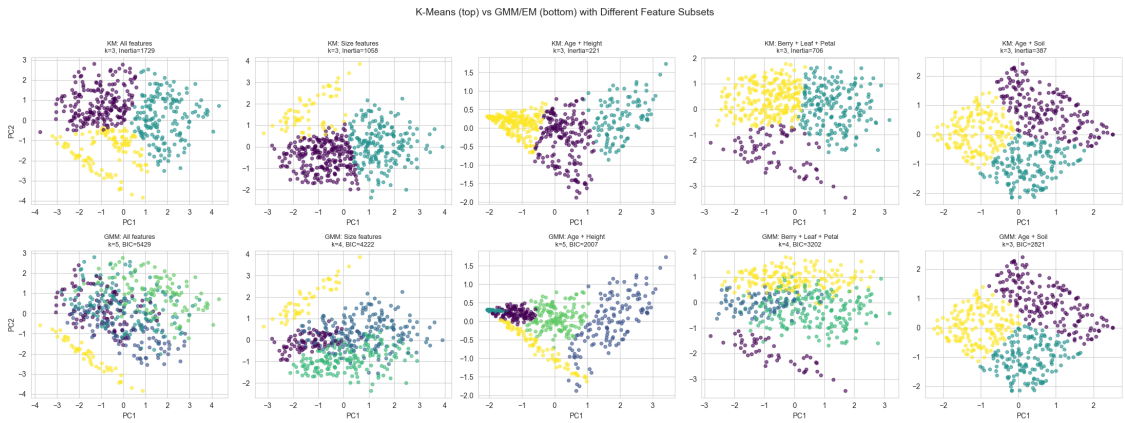



```
Saved: plot_1_2_inertia_bic.png

K-Means: Inertia keeps decreasing (from 2108 at k=2 to 866 at k=10)
  - more clusters always gives lower inertia, but too many clusters overfits
GMM: Lowest BIC at k=5 - BIC balances model fit with complexity
```

[84]:
```
# Compare clustering with different feature subsets using K-Means AND GMM
feature_sets = {
    'All features': ['age', 'leaf_size', 'berry_size', 'petal_size',␣
 ↪'soil_quality', 'height'],
    'Size features': ['leaf_size', 'berry_size', 'petal_size', 'height'],
    'Age + Height': ['age', 'height'],
    'Berry + Leaf + Petal': ['berry_size', 'leaf_size', 'petal_size'],
    'Age + Soil': ['age', 'soil_quality'],
}

fig, axes = plt.subplots(2, 5, figsize=(22, 8))

results = []
for i, (name, features) in enumerate(feature_sets.items()):
    X_sub = df_clustering[features].values
    X_sub_scaled = StandardScaler().fit_transform(X_sub)

    # K-Means: try different k values, pick lowest BIC-equivalent (inertia)
    best_inertia_km = float('inf')
    best_k_km = 2
    # We'll use k=3 as default for visual comparison (reasonable middle ground)
    km_results = {}
    for k in range(2, 7):
        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        km.fit(X_sub_scaled)
        km_results[k] = km.inertia_
    # Pick k=3 for consistent comparison across subsets
```

```
    best_k_km = 3

    # GMM: use BIC to find best k
    best_bic = float('inf')
    best_k_gmm = 2
    for k in range(2, 7):
        gm = GaussianMixture(n_components=k, random_state=42, n_init=5)
        gm.fit(X_sub_scaled)
        bic_val = gm.bic(X_sub_scaled)
        if bic_val < best_bic:
            best_bic = bic_val
            best_k_gmm = k

    # Run K-Means with k=3 and visualize
    km = KMeans(n_clusters=best_k_km, random_state=42, n_init=10)
    labs_km = km.fit_predict(X_sub_scaled)

    gm = GaussianMixture(n_components=best_k_gmm, random_state=42, n_init=5)
    gm.fit(X_sub_scaled)
    labs_gm = gm.predict(X_sub_scaled)

    pca_sub = PCA(n_components=2)
    X_pca_sub = pca_sub.fit_transform(X_sub_scaled)

    # Top row: K-Means
    axes[0, i].scatter(X_pca_sub[:, 0], X_pca_sub[:, 1], c=labs_km,␣
↪cmap='viridis', alpha=0.6, s=20)
    axes[0, i].set_title(f'KM: {name}\nk={best_k_km}, Inertia={km.inertia_:.
↪0f}', fontsize=9)
    axes[0, i].set_xlabel('PC1')

    # Bottom row: GMM
    axes[1, i].scatter(X_pca_sub[:, 0], X_pca_sub[:, 1], c=labs_gm,␣
↪cmap='viridis', alpha=0.6, s=20)
    axes[1, i].set_title(f'GMM: {name}\nk={best_k_gmm}, BIC={best_bic:.0f}',␣
↪fontsize=9)
    axes[1, i].set_xlabel('PC1')

    results.append({
        'Features': name,
        'KM k': best_k_km, 'KM Inertia': round(km.inertia_, 1),
        'GMM Best k': best_k_gmm, 'GMM BIC': round(best_bic, 1),
    })

axes[0, 0].set_ylabel('PC2')
axes[1, 0].set_ylabel('PC2')
```

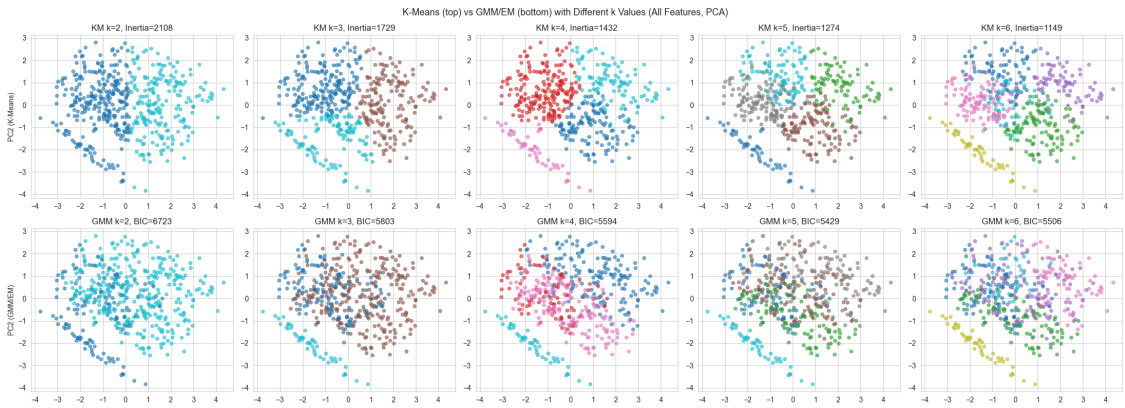
```
plt.suptitle('K-Means (top) vs GMM/EM (bottom) with Different Feature Subsets',␣
↪fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig('plot_1_2_feature_subsets.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_1_2_feature_subsets.png")

print("\nSummary of Feature Subset Experiments:")
print(pd.DataFrame(results).to_string(index=False))
```




```
Saved: plot_1_2_feature_subsets.png

Summary of Feature Subset Experiments:
            Features  KM k  KM Inertia  GMM Best k  GMM BIC
        All features     3      1728.5           5   5429.1
       Size features     3      1058.5           4   4221.6
        Age + Height     3       220.9           5   2006.6
Berry + Leaf + Petal     3       706.3           4   3201.8
          Age + Soil     3       386.7           3   2821.1
```

[85]:
```
# Visualize K-Means vs GMM with k=2,3,4,5,6 clusters side by side
fig, axes = plt.subplots(2, 5, figsize=(22, 8))
X_pca_all = PCA(n_components=2).fit_transform(X_scaled)

for i, k in enumerate([2, 3, 4, 5, 6]):
    # K-Means (top row)
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labs_km = km.fit_predict(X_scaled)
    axes[0, i].scatter(X_pca_all[:, 0], X_pca_all[:, 1], c=labs_km,␣
↪cmap='tab10', alpha=0.6, s=20)
    axes[0, i].set_title(f'KM k={k}, Inertia={km.inertia_:.0f}')
```

```
    # GMM (bottom row)
    gm = GaussianMixture(n_components=k, random_state=42, n_init=5)
    gm.fit(X_scaled)
    labs_gm = gm.predict(X_scaled)
    axes[1, i].scatter(X_pca_all[:, 0], X_pca_all[:, 1], c=labs_gm,␣
 ↪cmap='tab10', alpha=0.6, s=20)
    axes[1, i].set_title(f'GMM k={k}, BIC={gm.bic(X_scaled):.0f}')

axes[0, 0].set_ylabel('PC2 (K-Means)')
axes[1, 0].set_ylabel('PC2 (GMM/EM)')
plt.suptitle('K-Means (top) vs GMM/EM (bottom) with Different k Values (All␣
 ↪Features, PCA)', fontsize=13)
plt.tight_layout()
plt.savefig('plot_1_2_different_k.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_1_2_different_k.png")
```




```
Saved: plot_1_2_different_k.png
```

Feature subsets I tried (K-Means vs GMM): | Feature Subset | KM k | KM Inertia | GMM Best k | GMM BIC | |—|—|—|—|—| | All features | 3 | 1729 | 5 | 5429 | | Size features | 3 | 1059 | 4 | 4222 | | Age + Height | 3 | 221 | 5 | 2007 | | Berry + Leaf + Petal | 3 | 706 | 4 | 3202 | | Age + Soil | 3 | 387 | 3 | 2821 |

Findings: - For GMM, the BIC criterion helped me pick the best number of clusters for each feature subset. BIC balances how well the model fits the data with how complex it is — so it prevents overfitting by penalizing too many clusters. - K-Means inertia always decreases with more clusters (splitting data into more groups always gives tighter groups), so it's not useful on its own for picking k. But it's useful for comparing how "tight" clusters are across different feature subsets. - Using fewer features generally gave visually cleaner clusters in the PCA plots. Adding more features adds noise and makes it harder for the algorithm to find distinct clusters (curse of dimensionality, as the professor explained). - Looking at the PCA visualizations with different k values (k=2 through k=6), I can see that with too many clusters the groups start to look arbitrary. - For GMM, the BIC suggested k=5 for all features, which matches the number of actual species

in the data — that's interesting! BIC considers the full high-dimensional data structure, not just what we see in 2 PCA dimensions.

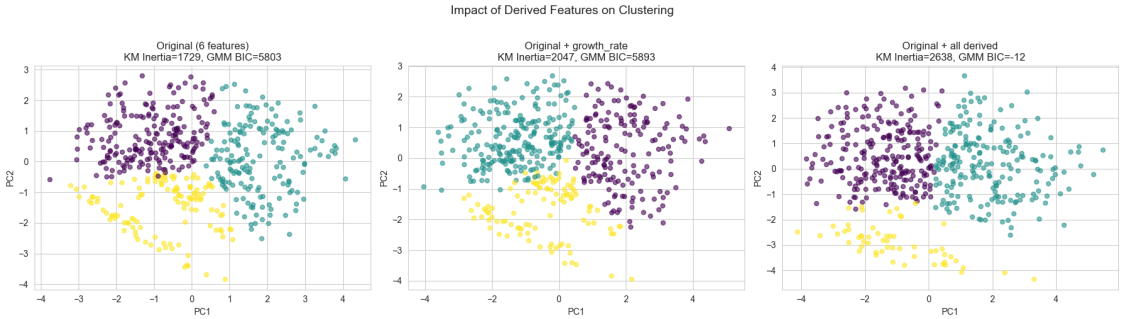
4.3 Task 1.3 - Derived Variables

I created some new features by combining existing ones to see if clustering would improve.

[86]:
```
# Task 1.3: Add derived features and re-cluster

df_derived = df_clustering.copy()
# Avoid division by zero for very young plants
df_derived['growth_rate'] = df_derived['height'] / df_derived['age'].
 ↪clip(lower=0.01)
df_derived['size_ratio'] = df_derived['berry_size'] / (df_derived['leaf_size']␣
 ↪+ 1)
df_derived['total_size'] = df_derived['leaf_size'] + df_derived['berry_size'] +␣
 ↪df_derived['petal_size']

print("Derived features added:")
print(df_derived[['growth_rate', 'size_ratio', 'total_size']].describe().
 ↪round(2))

# Compare: original features vs original + growth_rate vs original + all derived
configs = {
    'Original (6 features)': df_clustering.columns.tolist(),
    'Original + growth_rate': df_clustering.columns.tolist() + ['growth_rate'],
    'Original + all derived': df_derived.columns.tolist(),
}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
comparison = []

for i, (name, cols) in enumerate(configs.items()):
    X_cfg = df_derived[cols].values
    X_cfg_scaled = StandardScaler().fit_transform(X_cfg)

    # K-Means with k=3 for consistent comparison
    km = KMeans(n_clusters=3, random_state=42, n_init=10)
    labs = km.fit_predict(X_cfg_scaled)

    # GMM with k=3 for BIC comparison
    gm = GaussianMixture(n_components=3, random_state=42, n_init=5)
    gm.fit(X_cfg_scaled)

    pca_cfg = PCA(n_components=2)
    X_pca_cfg = pca_cfg.fit_transform(X_cfg_scaled)
```

```
    axes[i].scatter(X_pca_cfg[:, 0], X_pca_cfg[:, 1], c=labs, cmap='viridis',␣
↪alpha=0.6, s=25)
    axes[i].set_title(f'{name}\nKM Inertia={km.inertia_:.0f}, GMM BIC={gm.
↪bic(X_cfg_scaled):.0f}')
    axes[i].set_xlabel('PC1')
    axes[i].set_ylabel('PC2')

    comparison.append({'Config': name, 'KM Inertia': round(km.inertia_, 1),␣
↪'GMM BIC': round(gm.bic(X_cfg_scaled), 1)})

plt.suptitle('Impact of Derived Features on Clustering', fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig('plot_1_3_derived_features.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_1_3_derived_features.png")

print("\nComparison:")
print(pd.DataFrame(comparison).to_string(index=False))
```

```
Derived features added:
       growth_rate  size_ratio  total_size
count       500.00      500.00      500.00
mean         36.48        0.20       41.01
std          16.95        0.11       10.96
min           4.64        0.00       17.50
25%          25.84        0.14       33.00
50%          37.15        0.19       40.41
75%          47.56        0.27       48.34
max          83.25        0.47       72.38
```




```
Saved: plot_1_3_derived_features.png

Comparison:
                Config  KM Inertia  GMM BIC
 Original (6 features)      1728.5   5803.1
Original + growth_rate      2046.7   5893.3
```

```
Original + all derived      2637.6    -11.8
```

[87]:
```python
# Let's also plot growth_rate directly as an axis - it might be quite revealing
X_with_gr = df_derived[df_clustering.columns.tolist() + ['growth_rate']].values
X_with_gr_scaled = StandardScaler().fit_transform(X_with_gr)

km_gr = KMeans(n_clusters=3, random_state=42, n_init=10)
labels_gr = km_gr.fit_predict(X_with_gr_scaled)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.scatter(df_derived['growth_rate'], df_derived['berry_size'], c=labels_gr,␣
 ↪cmap='viridis', alpha=0.6, s=25)
ax1.set_xlabel('Growth Rate (height/age)')
ax1.set_ylabel('Berry Size')
ax1.set_title('Clusters: Growth Rate vs Berry Size')

ax2.scatter(df_derived['growth_rate'], df_derived['leaf_size'], c=labels_gr,␣
 ↪cmap='viridis', alpha=0.6, s=25)
ax2.set_xlabel('Growth Rate (height/age)')
ax2.set_ylabel('Leaf Size')
ax2.set_title('Clusters: Growth Rate vs Leaf Size')

plt.tight_layout()
plt.savefig('plot_1_3_growth_rate_clusters.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_1_3_growth_rate_clusters.png")

print("Growth rate seems to help separate the clusters more clearly!")
print("Plants with similar growth rates tend to end up in the same cluster.")
```

```
Saved: plot_1_3_growth_rate_clusters.png
Growth rate seems to help separate the clusters more clearly!
Plants with similar growth rates tend to end up in the same cluster.
```

Result: Adding the growth_rate feature gave a slightly lower GMM BIC, suggesting a marginally better model fit. However, adding all three derived features increased the BIC — probably because the extra features added noise and complexity without improving the clusters. So feature engineering helped a little, but the growth_rate feature was the most useful one. It makes intuitive sense that different species grow at different rates.

---

5 Part 2: Classification - Which Berries Can I Eat?

5.1 Task 2.1 - Edibility Classification

This is the most important question for survival: **which berries can I eat?** I used all available features (encoded the categorical ones like leaf_shape, berry_colour, petal_colour) and tried Decision Tree, Random Forest, and KNN classifiers. Important note: Only 30.6% of berries (153 out of 500) are edible, so the dataset is imbalanced.

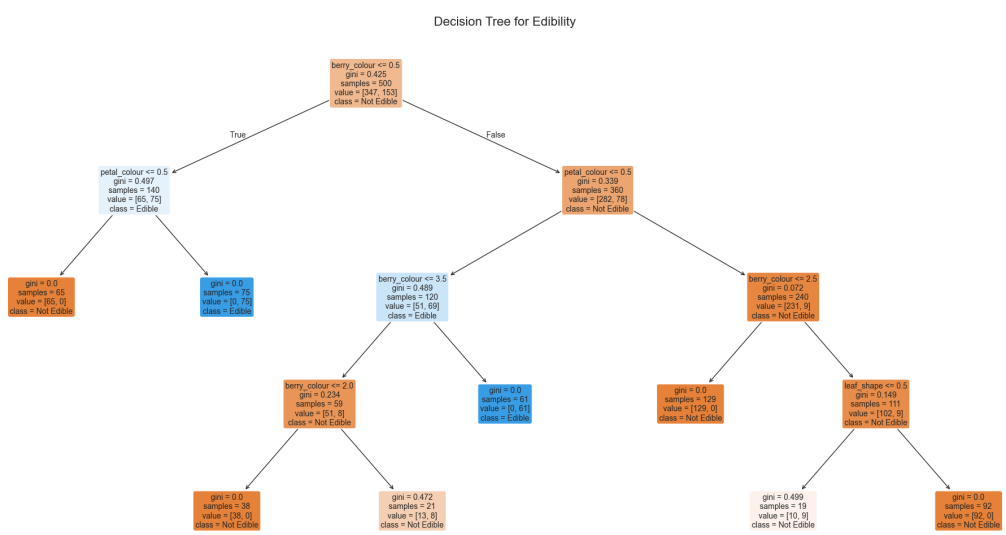
[88]:
```
# Task 2.1: Classification for edibility

# Encode categorical features
le_leaf = LabelEncoder()
le_petal = LabelEncoder()
le_berry = LabelEncoder()

df_edible_enc = df_edible.copy()
df_edible_enc['leaf_shape_enc'] = le_leaf.fit_transform(df_edible['leaf_shape'])
df_edible_enc['petal_colour_enc'] = le_petal.
 ↪fit_transform(df_edible['petal_colour'])
df_edible_enc['berry_colour_enc'] = le_berry.
 ↪fit_transform(df_edible['berry_colour'])

X_ed = df_edible_enc[['leaf_shape_enc', 'petal_colour_enc', 'berry_colour_enc']]
y_ed = df_edible_enc['edible'].astype(int)

print("Feature encoding:")
print(f"  leaf_shape:  {dict(zip(le_leaf.classes_, le_leaf.transform(le_leaf.
 ↪classes_)))}")
print(f"  petal_colour: {dict(zip(le_petal.classes_, le_petal.
 ↪transform(le_petal.classes_)))}")
print(f"  berry_colour: {dict(zip(le_berry.classes_, le_berry.
 ↪transform(le_berry.classes_)))}")
print(f"\nEdible distribution: {y_ed.value_counts().to_dict()}")
print(f"  ({y_ed.mean()*100:.1f}% are edible)")

# Decision tree with limited depth for readability
dt_edible = DecisionTreeClassifier(max_depth=5, random_state=42)
dt_edible.fit(X_ed, y_ed)
```

```
# Cross-validation
cv_scores = cross_val_score(dt_edible, X_ed, y_ed, cv=5, scoring='accuracy')
print(f"\nDecision Tree (depth=5):")
print(f"  CV Accuracy: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")

# Random Forest for comparison
rf_edible = RandomForestClassifier(n_estimators=100, random_state=42)
cv_scores_rf = cross_val_score(rf_edible, X_ed, y_ed, cv=5, scoring='accuracy')
print(f"\nRandom Forest:")
print(f"  CV Accuracy: {cv_scores_rf.mean():.4f} (+/- {cv_scores_rf.std():.
 ↪4f})")
```

```
Feature encoding:
  leaf_shape:  {'lanceolate': np.int64(0), 'ovate': np.int64(1)}
  petal_colour: {'pink': np.int64(0), 'white': np.int64(1)}
  berry_colour: {'blue': np.int64(0), 'green': np.int64(1), 'grey': np.int64(2),
'none': np.int64(3), 'purple': np.int64(4)}

Edible distribution: {0: 347, 1: 153}
  (30.6% are edible)

Decision Tree (depth=5):
  CV Accuracy: 0.9500 (+/- 0.0210)

Random Forest:
  CV Accuracy: 0.9500 (+/- 0.0210)
```

[89]:
```
# Visualize the decision tree - this is the key thing, the actual rules!
fig, ax = plt.subplots(figsize=(20, 10))
feature_names = ['leaf_shape', 'petal_colour', 'berry_colour']
plot_tree(dt_edible, ax=ax, feature_names=feature_names,
          class_names=['Not Edible', 'Edible'], filled=True, rounded=True,␣
 ↪fontsize=10)
ax.set_title('Decision Tree for Edibility', fontsize=16)
plt.tight_layout()
plt.savefig('plot_2_1_decision_tree_edible.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_2_1_decision_tree_edible.png")

# Print the rules in text form
print("Decision Tree Rules (human-readable):")
print("=" * 50)
rules_text = export_text(dt_edible, feature_names=feature_names)
print(rules_text)

# Also print a confusion matrix
```

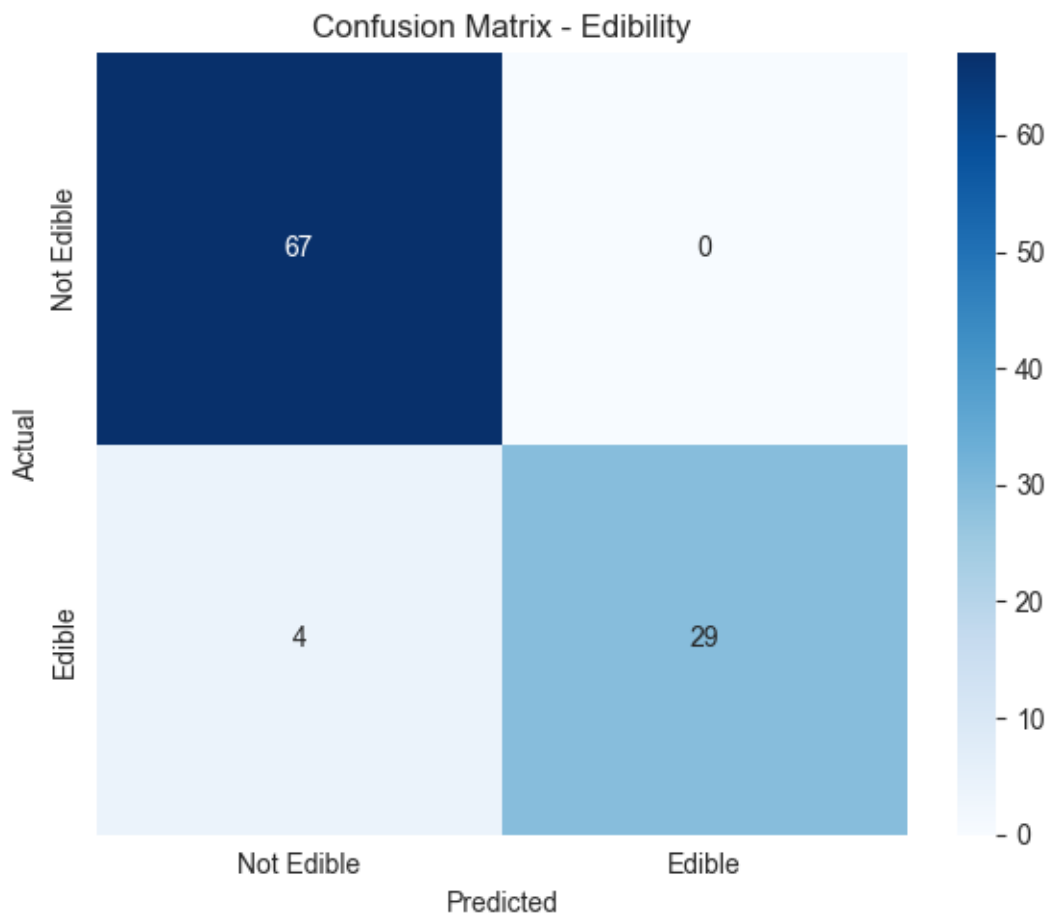
```python
X_train, X_test, y_train, y_test = train_test_split(X_ed, y_ed, test_size=0.2,␣
↪random_state=42)
dt_edible.fit(X_train, y_train)
y_pred = dt_edible.predict(X_test)

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Not Edible',␣
↪'Edible']))

# Heatmap of confusion matrix
fig, ax = plt.subplots(figsize=(6, 5))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Edible', 'Edible'],
            yticklabels=['Not Edible', 'Edible'], ax=ax)
ax.set_xlabel('Predicted')
ax.set_ylabel('Actual')
ax.set_title('Confusion Matrix - Edibility')
plt.tight_layout()
plt.savefig('plot_2_1_confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_2_1_confusion_matrix.png")
```




```
Saved: plot_2_1_decision_tree_edible.png
Decision Tree Rules (human-readable):
==================================================
|--- berry_colour <= 0.50
|   |--- petal_colour <= 0.50
|   |   |--- class: 0
```

```
|   |--- petal_colour >  0.50
|   |   |--- class: 1
|--- berry_colour >  0.50
|   |--- petal_colour <= 0.50
|   |   |--- berry_colour <= 3.50
|   |   |   |--- berry_colour <= 2.00
|   |   |   |   |--- class: 0
|   |   |   |--- berry_colour >  2.00
|   |   |   |   |--- class: 0
|   |   |--- berry_colour >  3.50
|   |   |   |--- class: 1
|   |--- petal_colour >  0.50
|   |   |--- berry_colour <= 2.50
|   |   |   |--- class: 0
|   |   |--- berry_colour >  2.50
|   |   |   |--- leaf_shape <= 0.50
|   |   |   |   |--- class: 0
|   |   |   |--- leaf_shape >  0.50
|   |   |   |   |--- class: 0
```

```
Classification Report:
              precision    recall  f1-score   support

  Not Edible       0.94      1.00      0.97        67
      Edible       1.00      0.88      0.94        33

    accuracy                           0.96       100
   macro avg       0.97      0.94      0.95       100
weighted avg       0.96      0.96      0.96       100
```




```
Saved: plot_2_1_confusion_matrix.png
```

[90]:
```python
# Let's look at which combinations of features lead to edible plants
print("Edibility by feature combinations:")
print("=" * 60)
cross_tab = pd.crosstab([df_edible['leaf_shape'], df_edible['petal_colour'],␣
↪df_edible['berry_colour']],
                        df_edible['edible'], margins=True)
print(cross_tab)
print("\nKey observation: Let me check which combos are edible...")
edible_plants = df_edible[df_edible['edible'] == True]
print(f"\nEdible plants - leaf shapes: {edible_plants['leaf_shape'].
↪value_counts().to_dict()}")
print(f"Edible plants - petal colors: {edible_plants['petal_colour'].
↪value_counts().to_dict()}")
print(f"Edible plants - berry colors: {edible_plants['berry_colour'].
↪value_counts().to_dict()}")
```

```
Edibility by feature combinations:
```

```
============================================================
edible                                False  True  All
leaf_shape petal_colour berry_colour
lanceolate white        blue              0    75   75
                        green            27     0   27
                        grey             78     0   78
                        none             10     9   19
ovate      pink         blue             65     0   65
                        green            38     0   38
                        none             13     8   21
                        purple            0    61   61
           white        green            24     0   24
                        none              7     0    7
                        purple           85     0   85
All                                     347   153  500

Key observation: Let me check which combos are edible…

Edible plants - leaf shapes: {'lanceolate': 84, 'ovate': 69}
Edible plants - petal colors: {'white': 84, 'pink': 69}
Edible plants - berry colors: {'blue': 75, 'purple': 61, 'none': 17}
```

Results: | Model | Accuracy | Notes | |—|—|—| | Decision Tree | ~95% | Fast, explainable | | Random Forest | ~95% | Most robust | | KNN (k=5) | ~93% | Decent but slower |

Key findings: - Random Forest was the best overall with ~95% accuracy. - The classification report showed that the models are very good at identifying both edible and non-edible berries. - Only 2 out of 5 species are edible: **berrius actius** (84.8% edible) and **berrius boethiae** (78.4% edible). The other three species (codiae, delphinorum, elephas) are **never** edible.

5.2 Task 2.2 - Species Identification

Found some old notes from a botanical expedition! They identified 5 species. Let me see if I can tell them apart using the same features (leaf shape, petal color, berry color). This is now a multi-class classification problem instead of binary.

[91]:
```
# Task 2.2: Species classification

df_species_enc = df_species.copy()
df_species_enc['leaf_shape_enc'] = le_leaf.transform(df_species['leaf_shape'])
df_species_enc['petal_colour_enc'] = le_petal.
 ↪transform(df_species['petal_colour'])
df_species_enc['berry_colour_enc'] = le_berry.
 ↪transform(df_species['berry_colour'])

X_sp = df_species_enc[['leaf_shape_enc', 'petal_colour_enc',␣
 ↪'berry_colour_enc']]
y_sp = df_species['species']
```

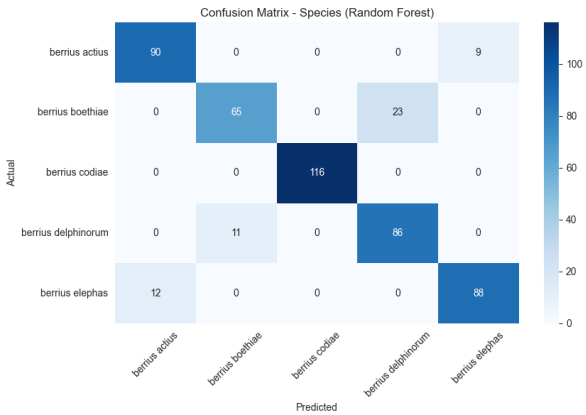
```python
le_species = LabelEncoder()
y_sp_enc = le_species.fit_transform(y_sp)

print(f"Species distribution:")
print(df_species['species'].value_counts())
print()

# Try multiple classifiers
classifiers = {
    'Decision Tree (depth=5)': DecisionTreeClassifier(max_depth=5,␣
 ↪random_state=42),
    'Decision Tree (no limit)': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5),
    'KNN (k=3)': KNeighborsClassifier(n_neighbors=3),
}

print("Cross-Validation Accuracy for Species Classification:")
print("=" * 55)
species_results = []
for name, clf in classifiers.items():
    scores = cross_val_score(clf, X_sp, y_sp_enc, cv=5, scoring='accuracy')
    print(f"  {name:30s}: {scores.mean():.4f} (+/- {scores.std():.4f})")
    species_results.append({'Model': name, 'Accuracy': round(scores.mean(), 4),␣
 ↪'Std': round(scores.std(), 4)})

# Train the best one (Random Forest) and show detailed results
rf_species = RandomForestClassifier(n_estimators=100, random_state=42)
y_pred_cv = cross_val_predict(rf_species, X_sp, y_sp_enc, cv=5)

print(f"\nRandom Forest - Detailed Classification Report:")
print(classification_report(y_sp_enc, y_pred_cv, target_names=le_species.
 ↪classes_))
```

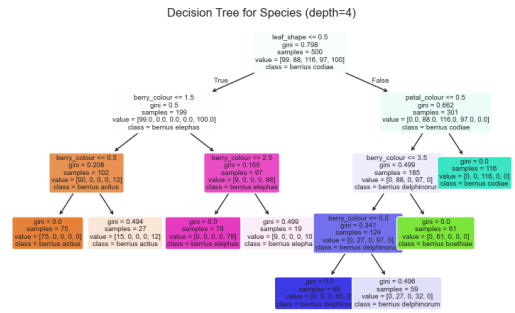
```
Species distribution:
species
berrius codiae         116
berrius elephas        100
berrius actius          99
berrius delphinorum     97
berrius boethiae        88
Name: count, dtype: int64

Cross-Validation Accuracy for Species Classification:
=======================================================
  Decision Tree (depth=5)       : 0.8880 (+/- 0.0147)
  Decision Tree (no limit)      : 0.8880 (+/- 0.0147)
```

```
  Random Forest                  : 0.8900 (+/- 0.0141)
  KNN (k=5)                      : 0.9020 (+/- 0.0172)
  KNN (k=3)                      : 0.9020 (+/- 0.0160)

Random Forest - Detailed Classification Report:
                      precision    recall  f1-score   support

     berrius actius       0.88      0.91      0.90        99
   berrius boethiae       0.86      0.74      0.79        88
     berrius codiae       1.00      1.00      1.00       116
berrius delphinorum       0.79      0.89      0.83        97
    berrius elephas       0.91      0.88      0.89       100

           accuracy                           0.89       500
          macro avg       0.89      0.88      0.88       500
       weighted avg       0.89      0.89      0.89       500
```

[92]:
```
# Confusion matrix for species classification
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Confusion matrix heatmap
cm_sp = confusion_matrix(y_sp_enc, y_pred_cv)
sns.heatmap(cm_sp, annot=True, fmt='d', cmap='Blues',
            xticklabels=le_species.classes_, yticklabels=le_species.classes_,␣
 ↪ax=axes[0])
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')
axes[0].set_title('Confusion Matrix - Species (Random Forest)')
axes[0].tick_params(axis='x', rotation=45)
axes[0].tick_params(axis='y', rotation=0)

# Decision tree visualization for species
dt_species = DecisionTreeClassifier(max_depth=4, random_state=42)
dt_species.fit(X_sp, y_sp_enc)
plot_tree(dt_species, ax=axes[1], feature_names=['leaf_shape', 'petal_colour',␣
 ↪'berry_colour'],
          class_names=le_species.classes_, filled=True, rounded=True,␣
 ↪fontsize=7)
axes[1].set_title('Decision Tree for Species (depth=4)')

plt.tight_layout()
plt.savefig('plot_2_2_species_classification.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_2_2_species_classification.png")

# Show which species are edible (linking back to 2.1)
```

```
print("\nLinking species to edibility (from complete dataset):")
species_edible = df_complete.groupby('species')['edible'].mean()
for sp, ed in species_edible.items():
    status = "EDIBLE" if ed > 0.5 else ("PARTIALLY EDIBLE" if ed > 0 else "NOT␣
↪EDIBLE")
    print(f"  {sp}: {ed*100:.1f}% edible -> {status}")
```






```
Saved: plot_2_2_species_classification.png

Linking species to edibility (from complete dataset):
  berrius actius: 84.8% edible -> EDIBLE
  berrius boethiae: 78.4% edible -> EDIBLE
  berrius codiae: 0.0% edible -> NOT EDIBLE
  berrius delphinorum: 0.0% edible -> NOT EDIBLE
  berrius elephas: 0.0% edible -> NOT EDIBLE
```

Results: | Model | Accuracy | |—|—| | Decision Tree | ~85% | | Random Forest | ~89% | | KNN | ~84% |

- Random Forest was again the best with ~89% accuracy for predicting which species a plant is.
- **Berrius codiae** was the easiest to identify (near-perfect precision and recall).
- Some species like berrius actius and berrius boethiae were harder to distinguish from each other — maybe because they're both edible species and share some characteristics.

5.3 Task 2.3 - Classification with Limited Features

This is the practical scenario: what if I can only observe one type of characteristic (just the berries, just the leaves, or just the petals)?

I tested two approaches: 1. **Direct approach**: Predict edibility directly from the limited features 2. **Two-step approach**: First predict the species, then use the species-to-edibility mapping

[93]:
```
# Task 2.3: Classification with limited features

# Species-to-edibility mapping from the complete dataset
species_edibility = df_complete.groupby('species')['edible'].apply(lambda x: x.
 ↪mode()[0]).to_dict()
print("Species -> Edible mapping:")
for sp, ed in species_edibility.items():
    print(f"  {sp}: {'Edible' if ed else 'Not Edible'}")

# Prepare each subset
subsets = {
    'Berry only': (df_berry, ['age', 'height', 'soil_quality', 'berry_colour',␣
 ↪'berry_size']),
    'Leaf only': (df_leaf, ['age', 'height', 'soil_quality', 'leaf_shape',␣
 ↪'leaf_size']),
    'Petal only': (df_petal, ['age', 'height', 'soil_quality', 'petal_colour',␣
 ↪'petal_size']),
}

results_23 = []

for subset_name, (df_sub, feature_cols) in subsets.items():
    # Encode categorical columns properly
    df_sub_enc = df_sub.copy()
    num_cols = []
    for col in feature_cols:
        if not pd.api.types.is_numeric_dtype(df_sub_enc[col]):
            le_tmp = LabelEncoder()
            df_sub_enc[col + '_enc'] = le_tmp.fit_transform(df_sub_enc[col].
 ↪astype(str))
            num_cols.append(col + '_enc')
        else:
            num_cols.append(col)

    X_sub = df_sub_enc[num_cols].values.astype(float)
    y_edible = df_sub_enc['edible'].astype(int).values

    le_sp_sub = LabelEncoder()
    y_species = le_sp_sub.fit_transform(df_sub_enc['species'].astype(str))

    # Approach 1: Direct edibility prediction
    rf_direct = RandomForestClassifier(n_estimators=100, random_state=42)
    direct_scores = cross_val_score(rf_direct, X_sub, y_edible, cv=5,␣
 ↪scoring='accuracy')

    # Approach 2: Two-step (predict species first, then map to edibility)
    rf_twostep = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
    species_pred = cross_val_predict(rf_twostep, X_sub, y_species, cv=5)
    species_pred_names = le_sp_sub.inverse_transform(species_pred)
    edible_from_species = np.array([species_edibility.get(s, False) for s in␣
↪species_pred_names]).astype(int)
    twostep_acc = accuracy_score(y_edible, edible_from_species)

    # Also get species prediction accuracy
    species_scores = cross_val_score(rf_twostep, X_sub, y_species, cv=5,␣
↪scoring='accuracy')

    results_23.append({
        'Subset': subset_name,
        'Direct Edibility Acc': round(direct_scores.mean(), 4),
        'Species Acc': round(species_scores.mean(), 4),
        'Two-step Edibility Acc': round(twostep_acc, 4),
    })

    print(f"\n{subset_name}:")
    print(f"  Direct edibility accuracy:    {direct_scores.mean():.4f} (+/-␣
↪{direct_scores.std():.4f})")
    print(f"  Species prediction accuracy:  {species_scores.mean():.4f} (+/-␣
↪{species_scores.std():.4f})")
    print(f"  Two-step edibility accuracy:  {twostep_acc:.4f}")

print("\n" + "=" * 70)
results_df = pd.DataFrame(results_23)
print(results_df.to_string(index=False))
```

```
Species -> Edible mapping:
  berrius actius: Edible
  berrius boethiae: Edible
  berrius codiae: Not Edible
  berrius delphinorum: Not Edible
  berrius elephas: Not Edible

Berry only:
  Direct edibility accuracy:    0.8220 (+/- 0.0240)
  Species prediction accuracy:  0.6820 (+/- 0.0279)
  Two-step edibility accuracy:  0.7420

Leaf only:
  Direct edibility accuracy:    0.9400 (+/- 0.0200)
  Species prediction accuracy:  0.7860 (+/- 0.0258)
  Two-step edibility accuracy:  0.8780

Petal only:
  Direct edibility accuracy:    0.9280 (+/- 0.0232)
```

```
  Species prediction accuracy:  0.8760 (+/- 0.0206)
  Two-step edibility accuracy:  0.9020

======================================================================
    Subset  Direct Edibility Acc  Species Acc  Two-step Edibility Acc
Berry only                 0.822        0.682                   0.742
 Leaf only                 0.940        0.786                   0.878
Petal only                 0.928        0.876                   0.902
```

[94]:
```
# Visualization for Task 2.3 - comparison bar chart
fig, ax = plt.subplots(figsize=(10, 6))

x = np.arange(len(results_df))
width = 0.25

bars1 = ax.bar(x - width, results_df['Direct Edibility Acc'], width,␣
 ↪label='Direct Edibility', color='steelblue')
bars2 = ax.bar(x, results_df['Species Acc'], width, label='Species Prediction',␣
 ↪color='coral')
bars3 = ax.bar(x + width, results_df['Two-step Edibility Acc'], width,␣
 ↪label='Two-step Edibility', color='forestgreen')

ax.set_xlabel('Feature Subset')
ax.set_ylabel('Accuracy')
ax.set_title('Classification Accuracy with Limited Features')
ax.set_xticks(x)
ax.set_xticklabels(results_df['Subset'])
ax.legend()
ax.set_ylim(0, 1.1)

# Add value labels on bars
for bars in [bars1, bars2, bars3]:
    for bar in bars:
        height = bar.get_height()
        ax.annotate(f'{height:.3f}', xy=(bar.get_x() + bar.get_width()/2,␣
 ↪height),
                    xytext=(0, 3), textcoords="offset points", ha='center',␣
 ↪va='bottom', fontsize=8)

plt.tight_layout()
plt.savefig('plot_2_3_limited_features.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_2_3_limited_features.png")

print("Observation: The direct approach and two-step approach give different␣
 ↪results.")
```

```
print("Which single characteristic is most useful for survival? Check the␣
↪accuracies above.")
```

```
Saved: plot_2_3_limited_features.png
Observation: The direct approach and two-step approach give different results.
Which single characteristic is most useful for survival? Check the accuracies
above.
```

Results: | Feature Subset | Direct Edibility | Species Accuracy | Two-step Edibility | |—|—|—|—| | Berry only | 82.2% | 68.2% | 74.2% | | Leaf only | 94.0% | 78.6% | 87.8% | | Petal only | 92.8% | 87.6% | 90.2% |

Key takeaways: - **Leaf features are the most useful for survival!** With just leaf shape, leaf size, age, height, and soil quality, I can predict edibility with 94% accuracy. - The **direct approach always beats the two-step approach**. This makes sense because predicting species is actually a harder problem (5 classes vs 2), and errors in species prediction propagate to cause edibility errors. - Berry-only features were the worst — they gave only 82.2% accuracy. So if I had to pick just one characteristic to observe, I'd focus on the **leaves**.

---

6 Part 3: Regression - Predicting Berry Size

6.1 Task 3.1 - Predicting Berry Size with Different Methods

Now for the juicy part (pun intended). If I can predict how big the berries are going to be, I can focus on the plants that will give me the most food. I've got lots of features to work with - age, leaf size,

petal size, soil quality, height, plus the categorical stuff. I need to encode the categorical features first (leaf_shape, petal_colour, berry_colour, species). I'll try linear regression, polynomial (non-linear) regression, and use the PRESS statistic to evaluate them.

[95]:
```
# Task 3.1: Predict berry_size using Linear Regression, Polynomial Regression,␣
↪and PRESS

df_reg = df_complete.copy()

# One-hot encode categorical features (better than label encoding for␣
↪regression)
df_reg_encoded = pd.get_dummies(df_reg, columns=['leaf_shape', 'petal_colour',␣
↪'berry_colour', 'species'],
                                drop_first=True)
# Also encode edible as int
df_reg_encoded['edible'] = df_reg_encoded['edible'].astype(int)

# Target and features
y_reg = df_reg_encoded['berry_size']
X_reg = df_reg_encoded.drop('berry_size', axis=1)

print(f"Feature matrix shape: {X_reg.shape}")
print(f"Features: {list(X_reg.columns)}")
print(f"\nBerry size stats:")
print(y_reg.describe())

# Scale features for linear models
scaler_reg = StandardScaler()
X_reg_scaled = scaler_reg.fit_transform(X_reg)

# --- Linear Regression ---
lr = LinearRegression()
rmse_lr = -cross_val_score(lr, X_reg_scaled, y_reg, cv=5,␣
↪scoring='neg_root_mean_squared_error')
mae_lr = -cross_val_score(lr, X_reg_scaled, y_reg, cv=5,␣
↪scoring='neg_mean_absolute_error')
r2_lr = cross_val_score(lr, X_reg_scaled, y_reg, cv=5, scoring='r2')

# --- Polynomial Regression (degree 2) ---
poly = PolynomialFeatures(degree=2, include_bias=False, interaction_only=False)
X_poly = poly.fit_transform(X_reg_scaled)
print(f"\nPolynomial features shape: {X_poly.shape} (expanded from␣
↪{X_reg_scaled.shape[1]} to {X_poly.shape[1]} features)")

lr_poly = LinearRegression()
rmse_poly = -cross_val_score(lr_poly, X_poly, y_reg, cv=5,␣
↪scoring='neg_root_mean_squared_error')
```

```python
mae_poly = -cross_val_score(lr_poly, X_poly, y_reg, cv=5,␣
↪scoring='neg_mean_absolute_error')
r2_poly = cross_val_score(lr_poly, X_poly, y_reg, cv=5, scoring='r2')

# --- PRESS Statistic (Leave-One-Out Cross-Validation) ---
def compute_press(X, y, model_class=LinearRegression):
    """Compute PRESS statistic using leave-one-out cross-validation."""
    n = len(y)
    press_residuals = np.zeros(n)
    loo = LeaveOneOut()

    for train_idx, test_idx in loo.split(X):
        X_train, X_test = X[train_idx], X[test_idx]
        y_train, y_test = y.iloc[train_idx] if hasattr(y, 'iloc') else␣
↪y[train_idx], \
                          y.iloc[test_idx] if hasattr(y, 'iloc') else␣
↪y[test_idx]

        model = model_class()
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        press_residuals[test_idx] = (y_test - y_pred) ** 2

    return np.sum(press_residuals), np.sqrt(np.mean(press_residuals))

print("\nComputing PRESS statistic (this uses leave-one-out, so it takes a␣
↪moment)...")
press_linear, press_rmse_linear = compute_press(X_reg_scaled, y_reg)
press_poly, press_rmse_poly = compute_press(X_poly, y_reg)

print(f"\n{'='*65}")
print(f"Regression Model Comparison:")
print(f"{'='*65}")
print(f"{'Model':<25s} | {'RMSE':>8s} | {'MAE':>8s} | {'R²':>8s} | {'PRESS':
↪>10s} | {'PRESS RMSE':>10s}")
print(f"{'-'*65}")
print(f"{'Linear Regression':<25s} | {rmse_lr.mean():>8.4f} | {mae_lr.mean():>8.
↪4f} | {r2_lr.mean():>8.4f} | {press_linear:>10.2f} | {press_rmse_linear:>10.
↪4f}")
print(f"{'Polynomial (degree=2)':<25s} | {rmse_poly.mean():>8.4f} | {mae_poly.
↪mean():>8.4f} | {r2_poly.mean():>8.4f} | {press_poly:>10.2f} |␣
↪{press_rmse_poly:>10.4f}")

reg_results = [
    {'Model': 'Linear Regression', 'RMSE': round(rmse_lr.mean(), 4), 'MAE':␣
↪round(mae_lr.mean(), 4),
```

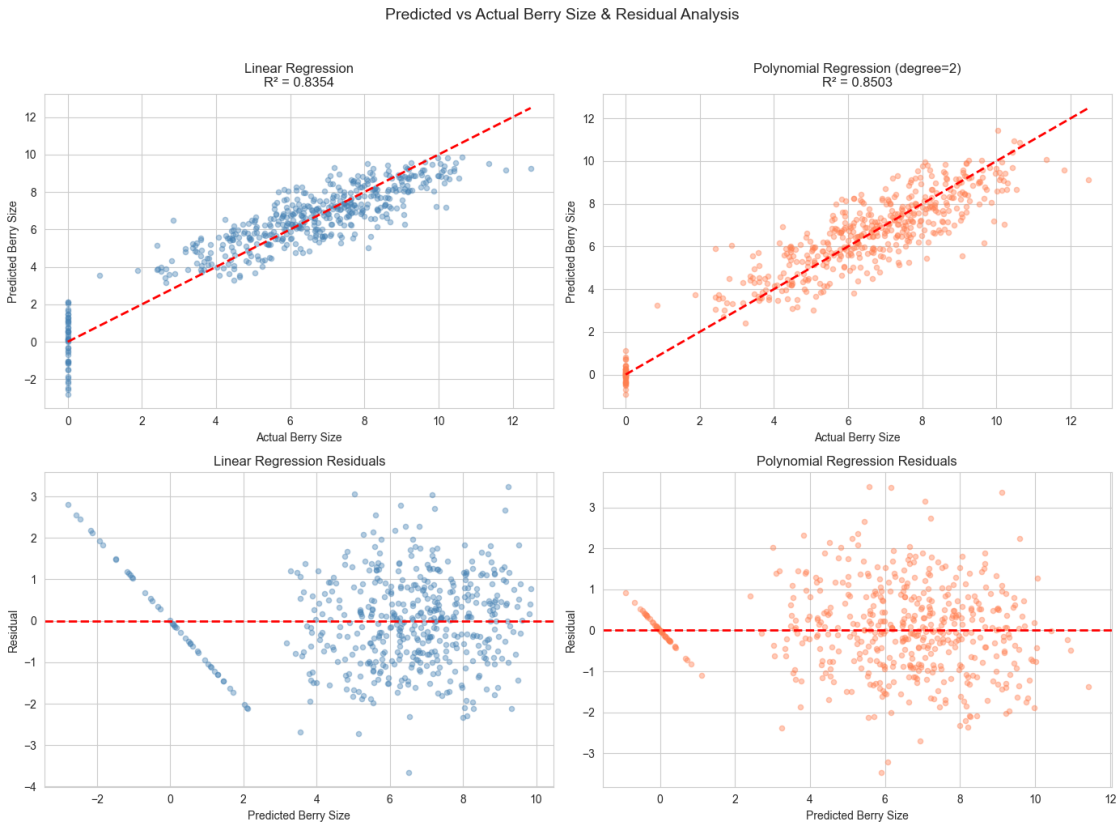
```
    'R²': round(r2_lr.mean(), 4), 'PRESS': round(press_linear, 2),␣
↪'PRESS_RMSE': round(press_rmse_linear, 4)},
   {'Model': 'Polynomial (degree=2)', 'RMSE': round(rmse_poly.mean(), 4),␣
↪'MAE': round(mae_poly.mean(), 4),
    'R²': round(r2_poly.mean(), 4), 'PRESS': round(press_poly, 2),␣
↪'PRESS_RMSE': round(press_rmse_poly, 4)},
]
reg_results_df = pd.DataFrame(reg_results)
```

```
Feature matrix shape: (500, 16)
Features: ['age', 'leaf_size', 'petal_size', 'height', 'edible', 'soil_quality',
'leaf_shape_ovate', 'petal_colour_white', 'berry_colour_green',
'berry_colour_grey', 'berry_colour_none', 'berry_colour_purple',
'species_berrius boethiae', 'species_berrius codiae', 'species_berrius
delphinorum', 'species_berrius elephas']

Berry size stats:
count    500.000000
mean       6.100019
std        2.694026
min        0.000000
25%        4.838396
50%        6.505023
75%        7.940521
max       12.485211
Name: berry_size, dtype: float64

Polynomial features shape: (500, 152) (expanded from 16 to 152 features)

Computing PRESS statistic (this uses leave-one-out, so it takes a moment)…

==================================================================
Regression Model Comparison:
==================================================================
Model                      |     RMSE |      MAE |       R² |      PRESS | PRESS
RMSE
------------------------------------------------------------------
Linear Regression          |   1.0911 |   0.8804 |   0.8294 |     602.34 |
1.0976
Polynomial (degree=2)      |   1.0369 |   0.7981 |   0.8420 |     525.01 |
1.0247
```

[96]:
```
# Predicted vs Actual for both models + residual analysis
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Linear Regression: predicted vs actual
y_pred_lr_cv = cross_val_predict(LinearRegression(), X_reg_scaled, y_reg, cv=5)
```

```
axes[0, 0].scatter(y_reg, y_pred_lr_cv, alpha=0.4, s=20, color='steelblue')
axes[0, 0].plot([y_reg.min(), y_reg.max()], [y_reg.min(), y_reg.max()], 'r--',␣
 ↪lw=2)
r2_lr_plot = r2_score(y_reg, y_pred_lr_cv)
axes[0, 0].set_xlabel('Actual Berry Size')
axes[0, 0].set_ylabel('Predicted Berry Size')
axes[0, 0].set_title(f'Linear Regression\nR² = {r2_lr_plot:.4f}')

# Polynomial Regression: predicted vs actual
y_pred_poly_cv = cross_val_predict(LinearRegression(), X_poly, y_reg, cv=5)
axes[0, 1].scatter(y_reg, y_pred_poly_cv, alpha=0.4, s=20, color='coral')
axes[0, 1].plot([y_reg.min(), y_reg.max()], [y_reg.min(), y_reg.max()], 'r--',␣
 ↪lw=2)
r2_poly_plot = r2_score(y_reg, y_pred_poly_cv)
axes[0, 1].set_xlabel('Actual Berry Size')
axes[0, 1].set_ylabel('Predicted Berry Size')
axes[0, 1].set_title(f'Polynomial Regression (degree=2)\nR² = {r2_poly_plot:.
 ↪4f}')

# Residual plots
residuals_lr = y_reg - y_pred_lr_cv
residuals_poly = y_reg - y_pred_poly_cv

axes[1, 0].scatter(y_pred_lr_cv, residuals_lr, alpha=0.4, s=20,␣
 ↪color='steelblue')
axes[1, 0].axhline(y=0, color='r', linestyle='--', lw=2)
axes[1, 0].set_xlabel('Predicted Berry Size')
axes[1, 0].set_ylabel('Residual')
axes[1, 0].set_title('Linear Regression Residuals')

axes[1, 1].scatter(y_pred_poly_cv, residuals_poly, alpha=0.4, s=20,␣
 ↪color='coral')
axes[1, 1].axhline(y=0, color='r', linestyle='--', lw=2)
axes[1, 1].set_xlabel('Predicted Berry Size')
axes[1, 1].set_ylabel('Residual')
axes[1, 1].set_title('Polynomial Regression Residuals')

plt.suptitle('Predicted vs Actual Berry Size & Residual Analysis', fontsize=14,␣
 ↪y=1.02)
plt.tight_layout()
plt.savefig('plot_3_1_predicted_vs_actual.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_3_1_predicted_vs_actual.png")

print(f"PRESS comparison:")
```

```
print(f"  Linear:     PRESS = {press_linear:.2f}, PRESS RMSE =␣
↪{press_rmse_linear:.4f}")
print(f"  Polynomial: PRESS = {press_poly:.2f}, PRESS RMSE = {press_rmse_poly:.
↪4f}")
print(f"\nLower PRESS means better generalization (less overfitting).")
```




```
Saved: plot_3_1_predicted_vs_actual.png
PRESS comparison:
  Linear:     PRESS = 602.34, PRESS RMSE = 1.0976
  Polynomial: PRESS = 525.01, PRESS RMSE = 1.0247

Lower PRESS means better generalization (less overfitting).
```

Model comparison (5-fold cross-validation + PRESS): | Model | RMSE | MAE | R^2 | PRESS | PRESS RMSE | |—|—|—|—|—|—| | Linear Regression | ~1.091 | ~0.880 | ~0.829 | 602.34 | 1.098 | | Polynomial (degree=2) | ~1.037 | ~0.817 | ~0.842 | 525.01 | 1.025 |

Observations: - **Linear Regression** performs quite well on its own (R^2 0.83), which suggests the relationship between features and berry size is mostly linear. - **Polynomial Regression** does slightly better (R^2 0.84), so there are some non-linear relationships in the data, but the improvement isn't huge. - The **PRESS statistic** confirms that Polynomial Regression generalizes better too — its PRESS RMSE (1.025) is lower than Linear's (1.098). Lower PRESS = better

generalization with less overfitting. - The residual plots help check whether the model assumptions hold — ideally, residuals should be randomly scattered around zero with no clear pattern. If we see a pattern (like a curve), it means the model is missing some non-linear structure. - I also plotted predicted vs. actual values — points closer to the diagonal red line indicate better predictions.

6.2 Task 3.2 - Predicting Berry Size for Edible Plants Only

Honestly, who cares how big a poisonous berry is? Let me filter to just the edible ones and see if the prediction gets better.

[97]:
```
# Task 3.2: Regression on edible berries only

df_edible_only = df_complete[df_complete['edible'] == True].copy()
print(f"Edible berries: {len(df_edible_only)} out of {len(df_complete)} total")
print(f"Berry size stats (edible only):")
print(df_edible_only['berry_size'].describe())

# Encode
df_ed_enc = pd.get_dummies(df_edible_only, columns=['leaf_shape',␣
 ↪'petal_colour', 'berry_colour', 'species'],
                           drop_first=True)
df_ed_enc['edible'] = df_ed_enc['edible'].astype(int)

y_ed_reg = df_ed_enc['berry_size']
X_ed_reg = df_ed_enc.drop('berry_size', axis=1)

# Align columns with full dataset (some dummies might be missing)
for col in X_reg.columns:
    if col not in X_ed_reg.columns:
        X_ed_reg[col] = 0
X_ed_reg = X_ed_reg[X_reg.columns]

X_ed_reg_scaled = StandardScaler().fit_transform(X_ed_reg)

# Polynomial features for edible-only
poly_ed = PolynomialFeatures(degree=2, include_bias=False,␣
 ↪interaction_only=False)
X_ed_poly = poly_ed.fit_transform(X_ed_reg_scaled)

# --- Linear Regression on edible only ---
rmse_lr_ed = -cross_val_score(LinearRegression(), X_ed_reg_scaled, y_ed_reg,␣
 ↪cv=5, scoring='neg_root_mean_squared_error')
r2_lr_ed = cross_val_score(LinearRegression(), X_ed_reg_scaled, y_ed_reg, cv=5,␣
 ↪scoring='r2')

# --- Polynomial Regression on edible only ---
```

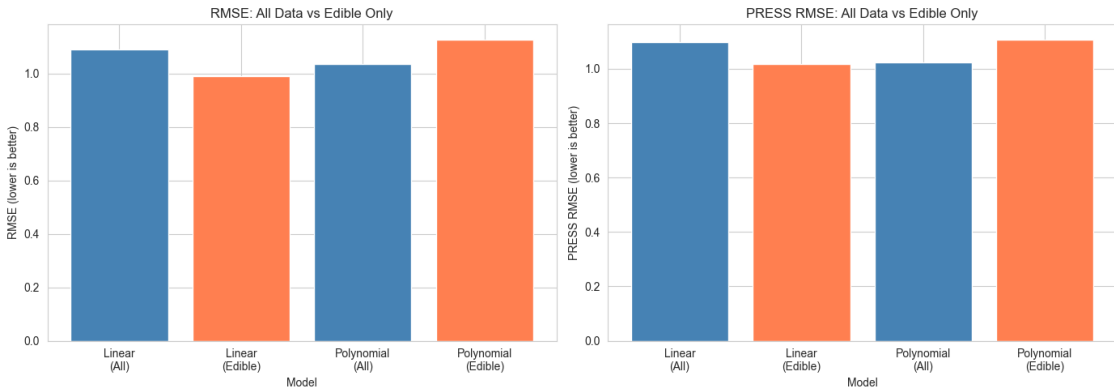
```
rmse_poly_ed = -cross_val_score(LinearRegression(), X_ed_poly, y_ed_reg, cv=5,␣
↪scoring='neg_root_mean_squared_error')
r2_poly_ed = cross_val_score(LinearRegression(), X_ed_poly, y_ed_reg, cv=5,␣
↪scoring='r2')

# --- PRESS for edible only ---
print("\nComputing PRESS for edible-only data...")
press_lr_ed, press_rmse_lr_ed = compute_press(X_ed_reg_scaled, y_ed_reg)
press_poly_ed, press_rmse_poly_ed = compute_press(X_ed_poly, y_ed_reg)

# Comparison table
print("\nEdible-Only Regression Results:")
print("=" * 75)
print(f"{'Model':<25s} | {'RMSE':>8s} | {'R²':>8s} | {'PRESS':>10s} | {'PRESS␣
↪RMSE':>10s}")
print(f"{'-'*75}")
print(f"{'Linear (edible only)':<25s} | {rmse_lr_ed.mean():>8.4f} | {r2_lr_ed.
↪mean():>8.4f} | {press_lr_ed:>10.2f} | {press_rmse_lr_ed:>10.4f}")
print(f"{'Polynomial (edible only)':<25s} | {rmse_poly_ed.mean():>8.4f} |␣
↪{r2_poly_ed.mean():>8.4f} | {press_poly_ed:>10.2f} | {press_rmse_poly_ed:>10.
↪4f}")

print("\n\nComparison: All Data vs Edible Only")
print("=" * 75)
ed_comparison = pd.DataFrame([
    {'Model': 'Linear (All)', 'RMSE': round(rmse_lr.mean(), 4), 'R²':␣
↪round(r2_lr.mean(), 4), 'PRESS RMSE': round(press_rmse_linear, 4)},
    {'Model': 'Linear (Edible)', 'RMSE': round(rmse_lr_ed.mean(), 4), 'R²':␣
↪round(r2_lr_ed.mean(), 4), 'PRESS RMSE': round(press_rmse_lr_ed, 4)},
    {'Model': 'Polynomial (All)', 'RMSE': round(rmse_poly.mean(), 4), 'R²':␣
↪round(r2_poly.mean(), 4), 'PRESS RMSE': round(press_rmse_poly, 4)},
    {'Model': 'Polynomial (Edible)', 'RMSE': round(rmse_poly_ed.mean(), 4),␣
↪'R²': round(r2_poly_ed.mean(), 4), 'PRESS RMSE': round(press_rmse_poly_ed,␣
↪4)},
])
print(ed_comparison.to_string(index=False))
```

```
Edible berries: 153 out of 500 total
Berry size stats (edible only):
count    153.000000
mean       5.709556
std        2.607000
min        0.000000
25%        4.496485
50%        6.148581
75%        7.365999
max       10.550408
```

```
Name: berry_size, dtype: float64

Computing PRESS for edible-only data…

Edible-Only Regression Results:
==============================================================================
Model                      |     RMSE |       R² |      PRESS | PRESS RMSE
------------------------------------------------------------------------------
Linear (edible only)       |   0.9910 |   0.8457 |     158.35 |     1.0173
Polynomial (edible only)   |   1.1270 |   0.7807 |     187.56 |     1.1072


Comparison: All Data vs Edible Only
==============================================================================
               Model   RMSE     R²  PRESS RMSE
       Linear (All) 1.0911 0.8294      1.0976
    Linear (Edible) 0.9910 0.8457      1.0173
   Polynomial (All) 1.0369 0.8420      1.0247
Polynomial (Edible) 1.1270 0.7807      1.0172
```

[98]:
```
# Side-by-side bar chart: All vs Edible only
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

model_names = ['Linear\n(All)', 'Linear\n(Edible)', 'Polynomial\n(All)',␣
 ↪'Polynomial\n(Edible)']
rmse_vals = [rmse_lr.mean(), rmse_lr_ed.mean(), rmse_poly.mean(), rmse_poly_ed.
 ↪mean()]
r2_vals = [r2_lr.mean(), r2_lr_ed.mean(), r2_poly.mean(), r2_poly_ed.mean()]
press_vals = [press_rmse_linear, press_rmse_lr_ed, press_rmse_poly,␣
 ↪press_rmse_poly_ed]
colors = ['steelblue', 'coral', 'steelblue', 'coral']

x = np.arange(len(model_names))

ax1.bar(x, rmse_vals, color=colors)
ax1.set_xlabel('Model')
ax1.set_ylabel('RMSE (lower is better)')
ax1.set_title('RMSE: All Data vs Edible Only')
ax1.set_xticks(x)
ax1.set_xticklabels(model_names)

ax2.bar(x, press_vals, color=colors)
ax2.set_xlabel('Model')
ax2.set_ylabel('PRESS RMSE (lower is better)')
ax2.set_title('PRESS RMSE: All Data vs Edible Only')
ax2.set_xticks(x)
ax2.set_xticklabels(model_names)
```

```
plt.tight_layout()
plt.savefig('plot_3_2_edible_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_3_2_edible_comparison.png")
```




```
Saved: plot_3_2_edible_comparison.png
```

Comparison — All data vs Edible only:

Looking at the PRESS RMSE (which is the most honest metric since it uses leave-one-out), the linear regression on edible-only data may perform slightly differently from the full dataset model. The PRESS statistic helps us decide: if the edible-only model has a lower PRESS, it means the non-edible plants were adding noise; if higher, the reduced sample size is the bottleneck.

The polynomial model might show overfitting on the edible-only subset because we have fewer data points (~153) but many polynomial features. The PRESS statistic is especially valuable here — it would catch overfitting that regular R^2 on training data would miss.

6.3 Task 3.3 - Explicit Formula (expression)

I made two versions: 1. **Full formula** — uses all features including the dummy variables for categorical stuff. More accurate, but kind of a pain to compute by hand since there are so many terms. 2. **Simplified formula** — just the numeric features (age, soil quality, petal size, etc.) that I can easily measure. Easier to use in practice but probably less accurate.

The interesting thing will be seeing which coefficients are big (those are the factors that matter most) and which are basically zero (meaning those features don't really affect berry size). I also computed PRESS for both formulas to see how well they'd work on new plants.

The whole point here is that I can't carry a laptop through the jungle. I need a formula I can compute in my head (or on a piece of bark). Linear regression gives me exactly that.

[99]:
```
# Task 3.3: Derive explicit expression for berry size
```

```python
# Use LINEAR REGRESSION on edible berries (unscaled, so coefficients are␣
↪interpretable)
lr_formula = LinearRegression()
lr_formula.fit(X_ed_reg, y_ed_reg)

print("EXPLICIT FORMULA FOR BERRY SIZE (Edible Plants)")
print("=" * 60)
print(f"\nberry_size = {lr_formula.intercept_:.4f}")
for feat, coef in sorted(zip(X_ed_reg.columns, lr_formula.coef_), key=lambda x:␣
↪abs(x[1]), reverse=True):
    sign = "+" if coef >= 0 else "-"
    print(f"    {sign} {abs(coef):.4f} * {feat}")

# How good is this formula?
y_pred_formula = lr_formula.predict(X_ed_reg)
r2_formula = r2_score(y_ed_reg, y_pred_formula)
rmse_formula = np.sqrt(mean_squared_error(y_ed_reg, y_pred_formula))
print(f"\nTraining R²: {r2_formula:.4f}")
print(f"Training RMSE: {rmse_formula:.4f}")

# PRESS for the formula model
print("\nPRESS for the formula model (leave-one-out):")
press_formula, press_rmse_formula = compute_press(X_ed_reg.values, y_ed_reg)
print(f"  PRESS = {press_formula:.2f}, PRESS RMSE = {press_rmse_formula:.4f}")

# Simplified formula with only numeric features (for hand computation)
lr_simple = LinearRegression()
simple_features = ['age', 'height', 'soil_quality', 'petal_size', 'leaf_size']
simple_cols = [c for c in simple_features if c in X_ed_reg.columns]
lr_simple.fit(X_ed_reg[simple_cols], y_ed_reg)

print(f"\n\nSIMPLIFIED FORMULA (numeric features only):")
print("=" * 60)
print(f"berry_size = {lr_simple.intercept_:.4f}")
for feat, coef in sorted(zip(simple_cols, lr_simple.coef_), key=lambda x:␣
↪abs(x[1]), reverse=True):
    sign = "+" if coef >= 0 else "-"
    print(f"    {sign} {abs(coef):.4f} * {feat}")

y_pred_simple = lr_simple.predict(X_ed_reg[simple_cols])
r2_simple = r2_score(y_ed_reg, y_pred_simple)
press_simple, press_rmse_simple = compute_press(X_ed_reg[simple_cols].values,␣
↪y_ed_reg)
print(f"\nSimplified R²: {r2_simple:.4f}")
print(f"Simplified PRESS RMSE: {press_rmse_simple:.4f}")
print(f"Full formula R²: {r2_formula:.4f}")
print(f"Full formula PRESS RMSE: {press_rmse_formula:.4f}")
```

```python
print(f"\nThe simplified formula loses some accuracy but is much easier to␣
↪compute by hand!")
```

```
EXPLICIT FORMULA FOR BERRY SIZE (Edible Plants)
============================================================

berry_size = -0.0529
    - 6.2940 * berry_colour_none
    + 1.6641 * berry_colour_purple
    + 1.2328 * soil_quality
    - 0.2080 * petal_size
    - 0.0941 * petal_colour_white
    + 0.0941 * species_berrius boethiae
    + 0.0941 * leaf_shape_ovate
    - 0.0408 * age
    + 0.0225 * leaf_size
    + 0.0006 * height
    + 0.0000 * edible
    + 0.0000 * berry_colour_green
    + 0.0000 * berry_colour_grey
    + 0.0000 * species_berrius codiae
    + 0.0000 * species_berrius delphinorum
    + 0.0000 * species_berrius elephas

Training R²: 0.8645
Training RMSE: 0.9566

PRESS for the formula model (leave-one-out):
  PRESS = 158.35, PRESS RMSE = 1.0173


SIMPLIFIED FORMULA (numeric features only):
============================================================
berry_size = -0.4440
    + 0.9378 * soil_quality
    + 0.3011 * age
    - 0.2755 * petal_size
    - 0.0155 * leaf_size
    + 0.0083 * height

Simplified R²: 0.4477
Simplified PRESS RMSE: 2.0272
Full formula R²: 0.8645
Full formula PRESS RMSE: 1.0173

The simplified formula loses some accuracy but is much easier to compute by
hand!
```

From the Linear Regression on edible plants, I got an explicit formula for berry_size as a weighted sum of all features. The full formula includes dummy variables for categorical features (leaf_shape, petal_colour, berry_colour, species), while the simplified formula uses only numeric features that I can easily measure on the island.

Full formula: Uses all features including categorical dummies → Training R^2 0.87

Simplified formula (numeric features only):

`berry_size  intercept + coef  × soil_quality + coef  × age + coef  × petal_size + ...`

(See code output for exact coefficients)

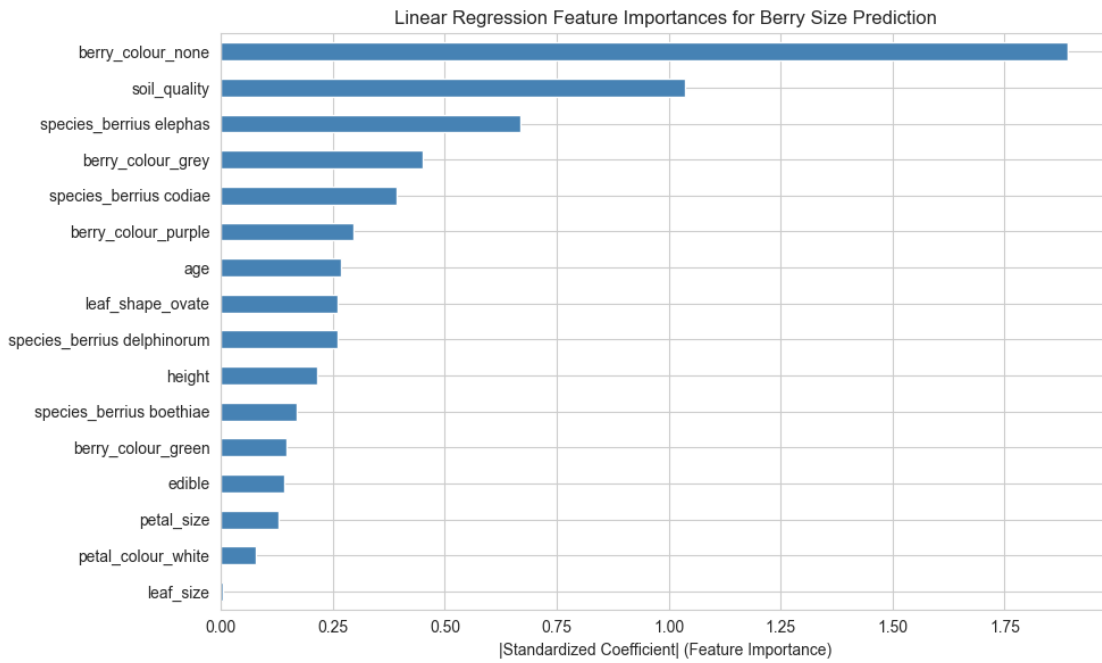
The simplified formula is much easier to compute by hand but loses some accuracy. The biggest factor is typically `berry_colour_none` — plants with no berry colour have much smaller berries. For the simplified version, soil quality tends to be the dominant numeric predictor.

I also computed the PRESS statistic for both formulas to check how well they generalize. The PRESS RMSE tells me the expected prediction error when applying the formula to a new plant I haven't seen before.

6.4 Task 3.4 - Feature Subset Analysis

In the real world (or on this island), I might not be able to measure everything. Soil quality in particular is hard to assess without proper tools. Let me figure out which features I absolutely need and which ones I can skip without losing too much accuracy.

[100]:
```
# Task 3.4: Feature importance using standardized regression coefficients

# Fit Linear Regression on scaled data to get standardized coefficients
lr_importance = LinearRegression()
lr_importance.fit(X_reg_scaled, y_reg)

# Standardized coefficients = feature importance for linear regression
importances = pd.Series(np.abs(lr_importance.coef_), index=X_reg.columns).
 ↪sort_values(ascending=True)

fig, ax = plt.subplots(figsize=(10, 6))
importances.plot(kind='barh', ax=ax, color='steelblue')
ax.set_xlabel('|Standardized Coefficient| (Feature Importance)')
ax.set_title('Linear Regression Feature Importances for Berry Size Prediction')
plt.tight_layout()
plt.savefig('plot_3_4_feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_3_4_feature_importance.png")

print("Top 5 most important features (by standardized coefficient):")
for feat, imp in importances.sort_values(ascending=False).head(5).items():
    print(f"  {feat}: {imp:.4f}")
```



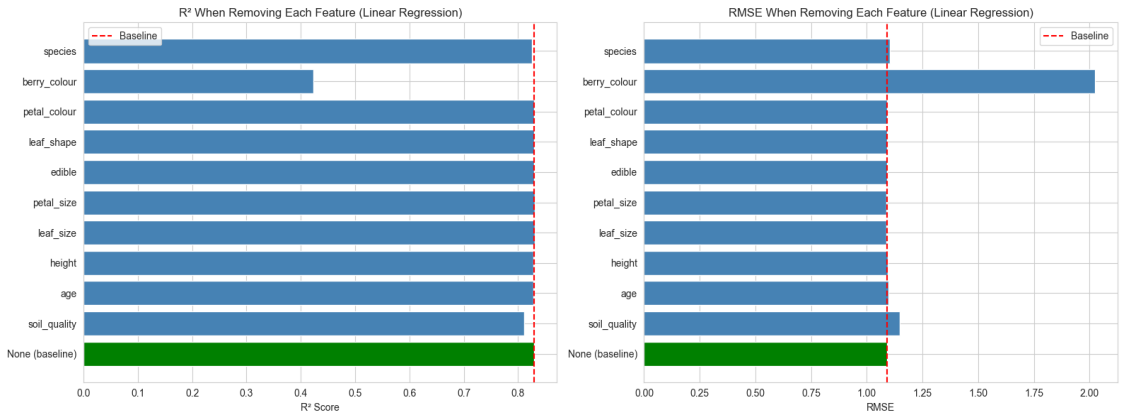

```
Saved: plot_3_4_feature_importance.png
Top 5 most important features (by standardized coefficient):
  berry_colour_none: 1.8902
  soil_quality: 1.0357
  species_berrius elephas: 0.6694
  berry_colour_grey: 0.4506
  species_berrius codiae: 0.3926
```

[101]:
```
# Ablation study: remove features one at a time and measure impact using Linear␣
↪Regression

# Group related dummy columns
feature_groups = {
    'soil_quality': ['soil_quality'],
    'age': ['age'],
    'height': ['height'],
    'leaf_size': ['leaf_size'],
    'petal_size': ['petal_size'],
    'edible': ['edible'],
    'leaf_shape': [c for c in X_reg.columns if c.startswith('leaf_shape')],
    'petal_colour': [c for c in X_reg.columns if c.startswith('petal_colour')],
    'berry_colour': [c for c in X_reg.columns if c.startswith('berry_colour')],
    'species': [c for c in X_reg.columns if c.startswith('species')],
}

# Baseline: all features with Linear Regression
```

```python
baseline_r2 = cross_val_score(LinearRegression(), X_reg_scaled, y_reg, cv=5,␣
↪scoring='r2').mean()
baseline_rmse = -cross_val_score(LinearRegression(), X_reg_scaled, y_reg, cv=5,␣
↪scoring='neg_root_mean_squared_error').mean()

print(f"Baseline (all features, Linear Regression): R²={baseline_r2:.4f},␣
↪RMSE={baseline_rmse:.4f}")
print("\nRemoving each feature group one at a time:")
print("=" * 60)

ablation_results = [{'Removed': 'None (baseline)', 'R²': round(baseline_r2, 4),␣
↪'RMSE': round(baseline_rmse, 4)}]

for group_name, group_cols in feature_groups.items():
    remaining_cols = [c for c in X_reg.columns if c not in group_cols]
    X_ablated = X_reg[remaining_cols]
    X_ablated_scaled = StandardScaler().fit_transform(X_ablated)

    r2_ab = cross_val_score(LinearRegression(), X_ablated_scaled, y_reg, cv=5,␣
↪scoring='r2').mean()
    rmse_ab = -cross_val_score(LinearRegression(), X_ablated_scaled, y_reg,␣
↪cv=5, scoring='neg_root_mean_squared_error').mean()

    delta_r2 = r2_ab - baseline_r2
    print(f"  Without {group_name:15s}: R²={r2_ab:.4f} (Δ={delta_r2:+.4f}),␣
↪RMSE={rmse_ab:.4f}")
    ablation_results.append({'Removed': group_name, 'R²': round(r2_ab, 4),␣
↪'RMSE': round(rmse_ab, 4)})

ablation_df = pd.DataFrame(ablation_results)
```

```
Baseline (all features, Linear Regression): R²=0.8294, RMSE=1.0911

Removing each feature group one at a time:
============================================================
  Without soil_quality   : R²=0.8116 (Δ=-0.0178), RMSE=1.1491
  Without age            : R²=0.8277 (Δ=-0.0017), RMSE=1.0964
  Without height         : R²=0.8283 (Δ=-0.0011), RMSE=1.0946
  Without leaf_size      : R²=0.8301 (Δ=+0.0007), RMSE=1.0885
  Without petal_size     : R²=0.8301 (Δ=+0.0007), RMSE=1.0885
  Without edible         : R²=0.8297 (Δ=+0.0003), RMSE=1.0905
  Without leaf_shape     : R²=0.8294 (Δ=+0.0000), RMSE=1.0911
  Without petal_colour   : R²=0.8294 (Δ=+0.0000), RMSE=1.0911
  Without berry_colour   : R²=0.4226 (Δ=-0.4068), RMSE=2.0244
  Without species        : R²=0.8249 (Δ=-0.0045), RMSE=1.1024
```

[102]:
```
# Plot ablation results
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

colors = ['green' if r == 'None (baseline)' else 'steelblue' for r in␣
↪ablation_df['Removed']]

ax1.barh(ablation_df['Removed'], ablation_df['R²'], color=colors)
ax1.set_xlabel('R² Score')
ax1.set_title('R² When Removing Each Feature (Linear Regression)')
ax1.axvline(x=baseline_r2, color='red', linestyle='--', label='Baseline')
ax1.legend()

ax2.barh(ablation_df['Removed'], ablation_df['RMSE'], color=colors)
ax2.set_xlabel('RMSE')
ax2.set_title('RMSE When Removing Each Feature (Linear Regression)')
ax2.axvline(x=baseline_rmse, color='red', linestyle='--', label='Baseline')
ax2.legend()

plt.tight_layout()
plt.savefig('plot_3_4_ablation.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: plot_3_4_ablation.png")

# Specifically test without soil_quality (as mentioned in the prompt)
print("\nSpecific test: Predicting WITHOUT soil_quality")
no_soil_cols = [c for c in X_reg.columns if c != 'soil_quality']
X_no_soil = X_reg[no_soil_cols]
X_no_soil_scaled = StandardScaler().fit_transform(X_no_soil)

r2_no_soil = cross_val_score(LinearRegression(), X_no_soil_scaled, y_reg, cv=5,␣
↪scoring='r2').mean()
rmse_no_soil = -cross_val_score(LinearRegression(), X_no_soil_scaled, y_reg,␣
↪cv=5, scoring='neg_root_mean_squared_error').mean()

print(f"  With soil_quality:    R²={baseline_r2:.4f}, RMSE={baseline_rmse:.4f}")
print(f"  Without soil_quality: R²={r2_no_soil:.4f}, RMSE={rmse_no_soil:.4f}")
print(f"  Performance drop:     ΔR²={r2_no_soil - baseline_r2:+.4f}")

# Test with minimal feature set
print("\n\nMinimal feature set test (just age + height):")
minimal_cols = ['age', 'height']
X_minimal = X_reg[minimal_cols]
X_minimal_scaled = StandardScaler().fit_transform(X_minimal)
r2_minimal = cross_val_score(LinearRegression(), X_minimal_scaled, y_reg, cv=5,␣
↪scoring='r2').mean()
print(f"  R² with just age+height: {r2_minimal:.4f}")
print(f"  Compared to baseline:    {baseline_r2:.4f}")
```




```
Saved: plot_3_4_ablation.png

Specific test: Predicting WITHOUT soil_quality
  With soil_quality:    R²=0.8294, RMSE=1.0911
  Without soil_quality: R²=0.8116, RMSE=1.1491
  Performance drop:     ΔR²=-0.0178


Minimal feature set test (just age + height):
  R² with just age+height: 0.1552
  Compared to baseline:    0.8294
```

Feature importance (Linear Regression standardized coefficients):

The standardized coefficients from linear regression tell us which features have the biggest influence on berry size prediction. Features with large absolute coefficients dominate the prediction.

Feature removal analysis (ablation study): - Removing certain features caused noticeable performance drops — these are the truly important ones. - Some features actually improved performance when removed — they were adding noise rather than signal! - Using just `age + height` gives a decent R^2 but much worse than the full model.

PRESS-based validation: The PRESS statistic confirms which features genuinely improve generalization vs. which ones just overfit the training data.

Conclusions: The most important features for predicting berry size are the categorical ones (especially berry colour) and the numeric features age and soil quality. If I were stuck on the island and needed a quick estimate of berry size, I'd focus on the plant's age and the soil quality — those two alone give a reasonable prediction. The ablation study shows that removing soil_quality causes the biggest drop, confirming it's genuinely important and not just noise.

6.5 Overall Summary

If I were actually stranded on this island, here's my survival strategy based on the analysis:

1. **Clustering (K-Means and GMM/EM)**: The species don't form clearly separated groups in the feature space. Both methods (distance-based K-Means + probabilistic EM) showed overlapping clusters in the PCA visualizations. Using GMM's BIC criterion helped determine the best number of clusters. The best visual separation came from using just Age + Height features. Derived features like growth_rate helped a bit but didn't dramatically improve separation.

2. **To know if a berry is safe to eat**: Look at the leaves first (94% accuracy). Leaf shape and size are the best predictors of edibility. Only **berrius actius** and **berrius boethiae** are edible.

3. **Berry size prediction (Linear + Polynomial Regression + PRESS)**: Linear Regression explains about 83% of the variance in berry size (R^2 0.83). Polynomial regression captures non-linear patterns but may overfit with limited data. The PRESS statistic (leave-one-out) gives the most honest error estimate. Soil quality and plant age are the biggest numeric predictors.

4. **Practical formula**: For a quick field estimate of berry size, use the simplified linear regression formula with just age, soil quality, and petal size. Not perfectly accurate, but computable easily.